



Beyond GNNs: a methodological benchmark of feature efficiency for link prediction in sparse developer networks

Cihan Bayraktar¹

Received: 2 September 2025 / Revised: 15 February 2026 / Accepted: 6 April 2026
© The Author(s) 2026

Abstract

This study presents a methodological investigation into the performance–efficiency trade-off of using classical feature-based models instead of graph neural networks (GNNs) for link prediction in sparse social networks. The study aims to systematically evaluate the effectiveness of engineered topological features compared to machine learning approaches in the context of GitHub developer collaborations. Using the MUSAE GitHub dataset (37,000 nodes, 289,000 links), we compare traditional machine learning models such as Logistic Regression, Random Forest, and LightGBM with modern GNN architectures such as Graph Convolutional Networks, GraphSAGE, and Graph Attention Networks. Our key finding is that, especially on sparse graphs, the LightGBM model, using rigorously engineered features (Common Neighbours, Jaccard Similarity, Adamic-Adar, Preferential Attachment, Node2Vec similarity), consistently outperforms standard GNN implementations (e.g., 99.3% accuracy and 0.9996 ROC-AUC in the ML community). These results challenge the tendency to automatically favour complex GNNs and provide powerful methodological insight that feature-based learning for sparse networks can deliver both high performance and computational efficiency. The main contribution of this work is to provide a rigorous and data-driven guide for model selection in graph-based learning and to challenge the automatic preference for GNNs in sparse networks. We also implemented a recommendation system prototype that serves as a practical demonstration of the methodological insights obtained.

Keywords Open-source collaboration · Social network analysis · Link prediction · Machine learning · Developer recommendation system

1 Introduction

This section presents the background to the study, including the rationale behind the chosen topic, an overview of current research in this area, the original research objective and the potential contributions of the proposed method.

✉ Cihan Bayraktar
cihanbayraktar@karabuk.edu.tr

¹ Department of Computer Technologies, Karabuk University, Karabuk, Turkey

1.1 Motivation

In recent years, the study of open-source software has contributed to a paradigm shift in which social interactions have become central to data analysis, as well as the technical contributions to the field. Online platforms such as GitHub, along with code hosting services, provide a social network structure that enables interaction between developers, facilitating the formation of collaborations and communities [23]. The relationships established between developers through follow, comment and contribution make these platforms' social aspects even stronger and support the formation of productive collaborative structures [16].

Today, GitHub is a social network that hosts more than 100 million work repositories, facilitating the development and management of technology projects with contributions from millions of developers [4]. Developers share more than just code on this social platform. They also select collaborators, contribute to projects they deem appropriate, follow each other's work, and provide feedback. Over time, this communication structure leads to the formation of a complex social network graph and contributes to the emergence of strong links between developers' productivity and their position in the network [22].

In this context, analysing social developer networks such as GitHub and integrating the information gathered from these networks into recommender systems is crucial to improving the developer experience and ensuring more efficient project contributions. Studies conducted for this purpose in the literature generally focus on link prediction, collaboration structure analysis, and project/store recommendation systems [34].

1.2 Related works

Thakrar & Chauhan [43] achieved successful outcomes using embedding-based graph features and GNN algorithms to develop a link recommendation system among developers on the GitHub Stargazers dataset. Similarly, Oliveira et al. [34] analysed heterogeneous developer relationships on GitHub, revealing the relationship between collaboration and software quality. Shao et al. [41] analysed article-repository matching and integrated established recommender systems into an architecture built using the GCN algorithm.

Similarly, Bai et al. [3] modelled heterogeneous resources on GitHub using a graph structure, proposing the AIPL system, which facilitates metapath-based information transfer and predicts links between issues and pull requests. This study found an increase in the performance of link prediction analyses of up to 20%. Alshara et al. [2] proposed a BIRCH-based intelligent system for the automatic prediction of missing PR-issue matches. The study stated that links could be recovered with high accuracy using the proposed system. Dello Vicario & Tortolini [10] analysed machine learning communities on GitHub. The analyses showed how technological orientations and collaboration structures can be revealed through network-based analysis. Resce et al. [37] measured the success of link predictions in academic collaboration networks. The study concluded that network-based features are stronger predictors than classical features. Lin et al. [24] used open-source developer-project interactions in a heterogeneous knowledge network-based model. The analysis achieved an increase in the accuracy rates of recommendation systems. Kosztyán et al. [19] performed link prediction analyses in open collaboration networks, testing the statistical accuracy of interactions within communities and established recommendation systems. Song et al. [42] conducted a study combining social network collaborations with corporate interests. The analyses performed using XGBoost, SVM and Random Forest algorithms revealed that the accuracy rates for link prediction had increased.

Although the aforementioned studies have analysed social interactions on GitHub from various perspectives, systems that focus directly on suggesting possible connections between developers, offering suggestions in both the community and personal spheres, are already being developed. For instance, Lazarine et al. [20] attempted to identify vulnerable developer clusters but did not implement recommendations. H. Zhou et al. [50] developed the GitEvolve system, which predicts project evolution but does not focus on predicting connectivity between developers.

Recent studies have seen the development of graph neural network-based approaches to link prediction problems. Lee et al. [21] achieved a 6.88% increase in accuracy by combining node and edge representations using early, intermediate, and late fusion strategies with their SFGCN model, proposing a powerful approach to the integration of multi-source data. Z. Zhou et al. [51] sought to mitigate uncertainty in the process of establishing missing connections by employing their information entropy-based IECNC model. This approach resulted in HITS@100 success rates of 94.68% and 96.44% on the Cora and CiteSeer datasets, respectively. In addressing the label imbalance issue, Mao et al. [31] employed PU-AUC optimization, thereby eliminating the necessity for class priority prediction in large networks and yielding efficient results. Positive-Unlabelled AUC (PU-AUC) is an evaluation metric recommended especially in scenarios where negative samples cannot be reliably labelled or where it is unclear whether unobserved links are truly negative. This approach aims to reduce the bias that may arise from treating all unobserved edges as negative. However, in this study, a balanced binary classification setup was created by explicitly sampling both positive and negative edges. Therefore, the classical ROC-AUC and average precision metrics offer a direct and reliable performance evaluation, thus the use of PU-AUC was not necessary.

The LHGNN + HA architecture developed by Rui et al. [39] has been shown to perform high-degree link predictions in hypergraph networks at a lower computational cost and to establish more accurate relationships between hyperconnections. Furthermore, the GCN-LSTM-based approach developed by Garompolo & Inzillo [12] has enhanced link prediction performance, particularly in dynamic SIoT networks, by comprehensively addressing temporal and spatial dependencies. This approach has attained 96.75% AUC and 95.02% MCC scores with a multi-head attention mechanism and dynamic feature planning. The study is distinguished by its sensitivity to time-varying social connection patterns, which have led to the identification of potential applications for temporal analysis of developer interactions on platforms such as GitHub. Finally, the local optimization policy (LOP) model developed by Nie et al. [33] optimized dynamic neighbourhood scopes for each node using deep reinforcement learning for graph neural networks and improved link prediction accuracy by capturing long-range dependencies with virtual nodes. This approach provides a powerful framework for customisable recommendation systems, particularly in low-connected and heterogeneous developer networks.

Despite the advanced GNN architectures developed in recent years, the performance—efficiency balance of these methods in practical applications has not been systematically examined. Although GNNs produce powerful representations thanks to their automatic feature learning capabilities, they come with high computational costs, data hunger, and scalability problems. Especially in scenarios where the graph is sparse and strong, manually designed topological features exist, the question of whether simpler, feature-based machine learning models can compete with, or even surpass, GNNs in terms of both accuracy and computational efficiency is critical. While most studies in the literature focus on accuracy improvement, methodological comparisons that holistically evaluate the performance—efficiency—scalability triangle are limited. This study aims not to propose a new architecture, but rather to provide a systematic benchmarking framework that empirically tests this balance.

Thus, it presents a data-driven and resource-saving model selection guide that explains which approach is more suitable and why in scenarios with specific graph features.

In recent years, link prediction problems have been addressed not only with classical similarity measures or Graph Neural Network architectures, but also with hybrid and factorization-based approaches relying on topological information integration. For example, Lu and Uddin [27] proposed a parametric model that defines node centralities and similarity measures as edge features and integrates them into a customized GNN layer, demonstrating that this approach provides superior performance compared to classical GCN and VGAE-based methods.

Similarly, the graph-regularized nonnegative matrix factorization approach proposed by Lv et al. [28], achieved competitive results, particularly in directional and temporal networks, by modelling PageRank centrality with global network information and graph arrangement terms with local structural information.

These studies demonstrate that the link prediction problem is not limited to deep learning architectures; centrality, similarity, and factorization-based methods also offer powerful alternatives. In this context, the present study adopts a comparative perspective encompassing different methodological families, offering a systematic evaluation of the performance–computational cost balance between classical machine learning and embedded GNN architectures.

The learning models used in this study were deliberately selected from among architectures commonly used and representative in the linkage estimation literature. The aim of the study is not to compete in performance with the most current or complex deep learning architectures; rather, it is to examine the relative effect of different feature representations (heuristic and embedding based) on learning algorithms within a controlled experimental framework. Therefore, model selection was limited to prioritize methodological comparability and experimental control.

1.3 Research objectives

The main objective of this study is to develop an intelligent recommendation system that can predict potential connections between open-source developers on the GitHub social network. This system will take into account both structural (topological) and semantic (feature based) attributes to provide collaboration recommendations at both the developer community and personalized levels based on these link predictions.

To this end, potential connections between developers were converted into numerical attributes using classical link prediction metrics (Common Neighbours, Jaccard Similarity, the Adamic-Adar Index, and Preferred Connection), as well as embedding representations obtained using the Node2Vec algorithm. Analyses were performed in a labelled environment using the MUSAE GitHub dataset obtained from the UCI machine learning repository, which contains social follow connections with more than 37,000 nodes. In this environment, developers were classified into two communities in the fields of web and machine learning (ML).

Different machine learning models, such as Logistic Regression, Random Forest, Light Gradient Boosting Machine (LightGBM), Graph Convolutional Network (GCN), GraphSAGE, and Graph Attention Network (GAT), were trained and comparatively evaluated based on these features. Based on the results obtained, a system was designed to provide real-time connection recommendations for the community as a whole and individual developers. This

system uses the LightGBM model, which performed the best. Thus, a decision-support mechanism aimed at increasing collaboration in open-source software development processes has been proposed.

1.4 Contributions

The principal contributions of this study are as follows:

- **A Methodological Insight into the Performance–Efficiency Trade-off:** This study provides critical analysis and experimental evidence of why well-tuned classical models (LightGBM) statistically significantly outperform standard GNN implementations (GCN, GraphSAGE, GAT) on large-scale, sparse graphs. This performance gap is attributed to the power of explicitly engineered topological features for link prediction, the difficulties GNNs face with extreme sparsity, and the higher computational load for pairwise prediction tasks.
- **A Comprehensive Empirical Benchmark:** We present a rigorous comparative analysis of classical machine learning models (LR, RF, LightGBM) and contemporary graph neural network architectures (GCN, GraphSAGE, GAT) for link prediction on the MUSAE GitHub dataset (a large-scale, real-world, and sparse developer social network).
- **Validation of a Hybrid Feature Engineering Approach:** We demonstrate that a feature set combining classical topological metrics (Common Neighbours, Jaccard Similarity, Adamic-Adar, Preferential Attachment) and Node2Vec embedding similarities can enable highly accurate link prediction.
- **Practical Validation with a Recommender System Prototype:** To demonstrate the real-world applicability of our methodological findings, we implemented a functional prototype system using the best-performing model. This prototype provides actionable collaboration recommendations and serves as a concrete validation of the performance–efficiency trade-offs identified in our benchmark.

The scope of this study is limited to sparse and inherently imbalanced developer networks observed in open-source software ecosystems. The GitHub network examined consists of real-world developer interactions with a low-density connection structure and a limited number of positive connections. How connection estimation performance might change in dense or balanced network structures is outside the direct scope of this study. The behaviour and generalizability of the method in such network structures will be systematically investigated in future studies.

The second part of the study explains the methods used, the feature engineering processes, the model configuration, the algorithms employed and the proposed approach model. The third section introduces the data set and provides the necessary information about the applied analyses. The fourth section presents the findings related to the learning algorithms created in the analysis processes, while the fifth section explains the recommendation system for the community and the personalized collaboration link recommendation module.

2 Methodology

This study aims to predict potential connections between web and machine learning developers on the GitHub social network, and to suggest pairs of developers who might collaborate well together. To this end, the GitHub multiscale attributed node embedding (MUSAE)

dataset, which was published in the UCI machine learning repository, was used to model social interactions between developers [38].

The original network structure has been divided into two subnetworks, labelled ‘web developers’ and ‘machine learning developers’, according to the developers’ areas of expertise. Sample groups consisting of developer pairs were created to train link prediction models on each subnetwork. These sample groups were structured to contain a balanced mix of positive examples with connections and negative examples without. The negative examples were obtained by randomly selecting pairs of developers without existing connections from the largest connected component of each subnetwork.

To create the learning models required for the classification process, the data was first subjected to feature engineering processes. At this stage, the following features were extracted for each developer pair:

- **Common Neighbours:** This effective and simple technique estimates the probability of a connection between two nodes in a social network structure by counting their shared connections. According to the explanation of this technique, the fact that two nodes follow or are connected to the same people increases the probability of a new connection between them [48].
- **Jaccard Similarity:** Jaccard similarity is used to calculate the similarity between the sets of neighbours of two nodes. It is defined as the ratio of the number of neighbours they have in common to the total number of neighbours they have. It is used to determine whether two nodes have similar connection structures [29].
- **Adamic-Adar Index:** The Adamic-Adar index focuses on the popularity of the neighbours that two nodes have in common. It evaluates connection probabilities according to the popularity levels of these neighbours. According to this metric, common neighbours with fewer connections carry greater weight in inverse proportion. Consequently, rare partnerships are considered more valuable [14].
- **Preferential Attachment:** This is based on the idea that nodes with a large number of connections (high degree) are more likely to generate new ones. This approach is generally preferred for modelling growth in social networks, based on the principle of ‘the rich get richer’ [36].
- **Node2Vec Embedding Similarity:** The Node2Vec algorithm is a random walk-based method used to convert the nodes in a network into vectors. The cosine similarity between these vectors provides an effective way of calculating the probability of connection between nodes. This method models structural similarities in more detail and surpasses traditional methods [13].

Feature extraction operations were performed on the dataset using the Common Neighbours, Jaccard Similarity, Adamic-Adar Index, and Preferential Attachment algorithms. Additionally, numerical samples specific to each node were extracted using the Node2Vec Embedding algorithm, and the cosine similarity value between node pairs was calculated and employed as a feature. The structural attributes extracted for each node were determined using nodes that were directly connected. For instance, node 0 is directly connected to nodes 1574, 3773, and 3571, among others. The extracted metrics were calculated separately for each pair of nodes, and vector representations were produced.

Once the feature extraction process was complete, resampling operations were applied to the node pairs. At this stage, positive (connected) and negative (unconnected) samples were created for both developer communities. Among web developers, for instance, the node pair (22134, 6146) was labelled a positive example (1), and the node pair (1937, 37044) a

negative example (0). These sampling procedures were applied to create balanced datasets to optimise the performance of link prediction using learning models.

Once the feature extraction process had been completed, data were generated for the classification process using the Logistic Regression (LR), Random Forest (RF), and Light Gradient Boosting Machine (LightGBM) machine learning algorithms, as well as the Graph Convolutional Network (GCN) algorithm. The models produced by each algorithm were trained separately for the web and machine learning subnetworks, and the results were tested.

2.1 Logistic regression (LR)

LR is a linear algorithm that is used extensively in binary classification analyses. It generates probabilities for all classes by modelling the potential nonlinear relationship between the input data and the result using a sigmoid function. For this reason, it is particularly useful in classification analyses of a presence/absence nature. LR is one of the most popular algorithms for social network analysis due to its interpretability and computational efficiency [9].

The most significant advantage of the algorithm is its high interpretability. Model coefficients directly demonstrate the impact of variables in classification analysis. For this reason, LR has been favoured in studies conducted in many branches of science. Additionally, its low computational cost makes it ideal for rapid model development in large datasets [7].

Despite being a frequently preferred algorithm, LR also has various limitations. Notably, its performance deteriorates in cases involving multiple linear connections, and it is inadequate for modelling nonlinear decision boundaries. In such cases, it may perform worse than tree-based algorithms. Recent studies have observed that hybrid structures combining LR with more advanced algorithms are being used to solve nonlinear problems [46].

2.2 Random forest (RF)

RF is an ensemble learning algorithm that stands out due to its high-accuracy rate, low variance, and resistance to overlearning in regression and classification analyses. Essentially, it is based on the principle of randomly dividing the dataset into subsets and training each one with a decision tree. More than one decision tree is formed, and the final prediction is determined by a majority vote among the results obtained from these trees. The randomization rule can therefore be applied at the point of training and feature selection. This structure enables the model to make stronger predictions and reduce the risk of overlearning [6].

RF is a popular algorithm in various fields, such as link prediction, traffic prediction, and user behaviour analysis in social network analysis. Its strong capacity to cope with missing data and superior performance in statistical metric results in classification and regression analyses make it widely used in analyses [44].

The limitations of the algorithm are difficult to interpret, as it is an ensemble model consisting of a combination of decision trees. Additionally, when used on large datasets, the computational costs are high due to the requirement to train a large number of trees. Nevertheless, it is considered an effective classification algorithm, providing important services such as feature importance ranking and successfully coping with imbalanced datasets [30].

2.3 Light gradient boosting machine (LightGBM)

It is a gradient-boosting-based decision tree algorithm developed by Microsoft. Designed to overcome the performance limitations of the XGBoost algorithm, LightGBM attracts attention with its high performance and low computational cost, particularly in large-scale data analysis. LightGBM's main difference from other tree-based algorithms is that it decomposes the samples in the dataset using a histogram-based approach and grows decision trees based on leaves. This significantly reduces learning time and increases accuracy [17].

LightGBM has achieved notable success in datasets such as link prediction, fraud detection, and disease diagnosis, particularly in datasets with class imbalance. This feature makes it the ideal algorithm for solving difficult problems such as medical data analysis and social network analysis. Numerous studies demonstrate that LightGBM outperforms other classical algorithms [32].

However, optimizing the hyperparameters of the LightGBM algorithm is delicate and requires caution. In terms of interpretability, it is inferior to LR or one-dimensional decision trees [35].

2.4 Graph convolutional networks (GCN)

A GCN is a deep learning algorithm that combines node-based features and network structure to perform learning. Unlike classical neural network algorithms, GCN updates features by obtaining information from neighbouring nodes in each layer. This allows both the content and the topological structure to be modelled together. GCNs are effectively used for graph-based data, such as in social network analysis, biological networks, commercial relationships, and information networks [18].

The fundamental benefit of GCNs in data analysis studies is their ability to naturally model relationships between nodes [8]. Successful results and high performance have recently been reported in various studies, including traffic prediction [44], fraud detection [11], social network analysis [46], and trade flow modelling [40].

Despite its advantages, the GCN algorithm has some limitations. Notably, it incurs high computational costs for large-scale, sparse graphs and can encounter complications during the learning process, such as overfitting. Furthermore, the algorithm's learning performance significantly declines when there are deficiencies or errors in the network structure. For this reason, many studies have integrated the GCN algorithm with random walks, attention mechanisms, and variational methods to enhance its performance [47].

2.5 GraphSAGE

GraphSAGE is a sampling-based graph neural network architecture developed to learn node embeddings on large-scale graphs, suitable for supervised and semi-supervised learning. In contradistinction to conventional graph neural networks, GraphSAGE samples local neighbourhood information for each node as opposed to loading the entire graph into memory, and employs the features obtained from this subgraph to produce node representations. This configuration facilitates scalable learning of inter-node relationships, particularly in dynamic and large graphs [15].

GraphSAGE is used in various applications, including link prediction and node classification. In this study, the applicability of GraphSAGE is primarily demonstrated in social network

analysis and recommendation systems, as evidenced by the GitHub developer network experiments presented in Sect. 3 and the real-time recommendation system prototype described in Sect. 5. Bhatkar et al. [5], in their groundbreaking work, used the GraphSAGE architecture to perform link prediction on real-world social network data. Their study reported a significant increase in accuracy compared to traditional methods. Afoudi et al. [1], further developed the GraphSAGE architecture, proposing an enhanced structure that improves accuracy and stability in recommendation systems.

The primary benefit of GraphSAGE lies in its capacity to generalise on newly observed nodes and its computational efficiency when employing mini-batch training. However, it is important to note that the model is not without its limitations. These include a sensitivity to hyperparameter selection and a limited capacity for modelling of global graph structures [1, 5].

2.6 Graph attention network (GAT)

GAT employs an attention mechanism that is based on the learning of the importance of information from each node's neighbours to learn node representations. This approach is predicated on the recognition that, in the context of the aforementioned problem, it is possible to distinguish between neighbours and to attribute to each one a contribution, with the weight of this contribution being different for each individual. In contrast to GCN methods, it offers a more flexible and sensitive modelling approach with learnable weights, as opposed to fixed weights [49].

GATs have been shown to excel in link prediction and node classification tasks. A comparative study by Verma et al. [45] demonstrated that GAT models exhibited superior performance in comparison to alternative methods on various benchmark datasets, particularly in the context of semi-supervised node classification tasks involving limited labelled data. GAT architectures have been demonstrated to be efficacious in the modelling of heterogeneous node structures. Liu et al. [26] successfully learned functional connection structures by integrating the GAT architecture into a node-feature-based structure.

Nevertheless, a critical constraint imposed by GAT models pertains to their scalability. In dense and large graph structures, the process of collecting information from a large number of neighbours can become costly. Furthermore, given that GAT models are oriented towards local neighbourhoods, they may be deficient in their capacity to model long-range dependencies. The global–local graph attention mechanism, proposed by K. Lin et al. [25], was developed to overcome this limitation and achieve better results.

Generally speaking, GAT facilitates the contextual evaluation of node interactions and is employed as an effective classifier in a variety of domains, including social networks, recommendation systems, and open-source collaboration analyses.

2.7 Proposed approach

As part of the study, data analysis was performed to predict potential work teams among web and machine learning developers on the GitHub system and to create a recommendation system. Learning models were created using machine learning algorithms such as Logistic Regression, Random Forest, and Light Gradient Boosting Machine, as well as the deep learning algorithm Graph Convolutional Network, GraphSAGE, and Graph Attention Network, during the creation of the appropriate recommendation system, and the results were

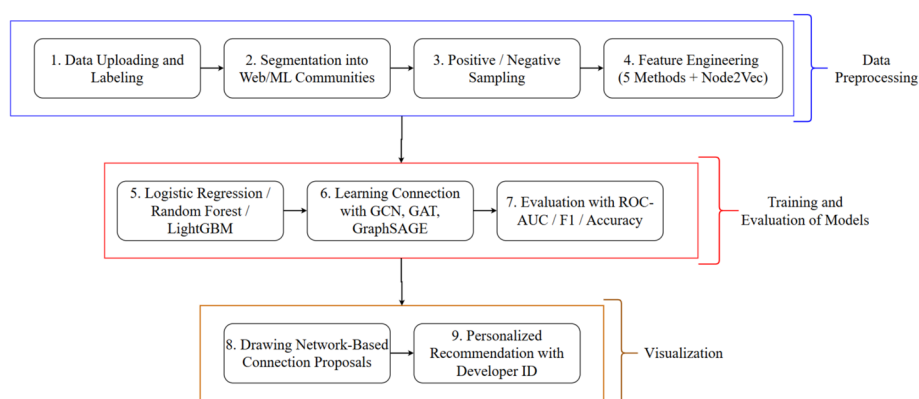


Fig. 1 System architecture of proposed model

compared. The processes carried out within the scope of the proposed approach are shown in Fig. 1.

The proposed model consists of four basic processes: data preprocessing, feature engineering, the learning process and recommendation generation. First, the nodes representing developers in the MUSAE dataset were divided into two subnetworks (Web and Machine Learning) according to the target label information. Including edge information representing follow interactions within the network, along with features extracted from developer profiles, created a meaningful graph structure capable of predicting connections for both communities. Within this structure, existing connections were accepted as positive examples, while node pairs without connections but located within the same connected component were randomly sampled as negative examples.

To prevent data leakage, feature extraction was performed solely from the training graph after splitting the training and test data. Positive correlations from the test set were removed from the graph, and features based on Common Neighbours, Jaccard Similarity, Adamic-Adar, Preferential Attachment, and Node2Vec were calculated without using test correlation information. This prevented the test data from indirectly affecting the features and ensured a fair and methodologically consistent evaluation process. Furthermore, the experiments were conducted in 20 independent replicates; in each replicate, the data splitting and negative sampling process was reapplied using different random seed values, and the results were reported as mean $\pm$ standard deviation.

In the second stage of the model, the feature engineering process involved creating a feature vector specific to each node pair. Classical link prediction metrics were employed to demonstrate the structural relationships between the features within the generated vectors. Additionally, the cosine similarity measure between the embedding vectors obtained using the Node2Vec algorithm was included in the model analysis as a feature. Thus, each pair of nodes was transformed into a vector containing five features, which were analyzed along with their structural and relational characteristics. These feature vectors were then used as input data for classification processes.

The third step involved training models capable of classifying the generated node pair vectors as connected or not connected. Logistic Regression, Random Forest, Light Gradient Boosting Machine, and Graph Convolutional Network algorithms were used in this process.

Following training and evaluation of the learning models, the appropriate model was selected for the final stage.

The fourth and final stage involved generating new connection suggestions for each developer using the trained model. These suggestions were ranked according to the positive class output predicted by the model with a high probability and visualized on the network. The aim was to identify developers with high collaboration potential within the communities.

The multi-stage system proposed in this study provides a practical framework for link prediction by combining hand-crafted topological and semantic features. It demonstrated high accuracy on the MUSAE dataset, though its performance on other network types or temporal data requires further validation.

In recent years, Graph Neural Network-based methods developed for link prediction problems have made significant progress. Particularly in the last three years, more complex architectures have been proposed, including multilayer fusion approaches, attention mechanisms, hypergraph structures, and reinforcement learning-based neighbourhood optimizations. This study comprehensively examines the current state of affairs by including recent works such as Lee et al. [21], Zhou et al. [51], Mao et al. [31], Rui et al. [39], and Nie et al. [33] in the literature review.

However, the primary aim of this study is not to propose a new state-of-the-art (SOTA) model or to conduct a comprehensive competition with all modern GNN variants. The goal is to compare the performance–efficiency balance of the most commonly used representative classical machine learning and basic GNN architectures under the same conditions. Therefore, experimental comparisons are limited to standard and basic architectures widely used in the literature. The investigation of more complex and computationally expensive next-generation GNN architectures will be considered in future studies.

2.8 Evaluation metrics

In this study, the link prediction problem is addressed as a supervised binary classification problem with explicit sampling of positive and negative edges. Therefore, commonly used classification-based performance metrics from the literature were preferred in the evaluation process. Precision, Recall, and F1-score metrics measure class-based prediction quality; Accuracy measures overall accuracy, and ROC-AUC measures threshold-independent discrimination performance.

Alternative metrics proposed in recent years, such as PU-AUC, NDCG, or Hits@k, are generally designed for positive-unlabelled or ranking-based prediction scenarios. Since negative examples are explicitly defined in our study and the aim is to directly compare binary classification performance, the five selected metrics provide a sufficient and directly comparable evaluation framework.

The mathematical definitions of the evaluation criteria used are given below:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

$$F1 - \text{Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (4)$$

ROC-AUC: Expresses the probability that a randomly selected positive sample will score higher than a randomly selected negative sample.

In the equations above, *TP* represents true positives (those correctly predicted as having a real connection), *FP* represents false positives (those predicted as having a connection when there is no connection), *TN* represents true negatives, and *FN* represents false negatives.

3 Experimental studies

3.1 Definition of data

This study used the GitHub MUSAE dataset, obtained from the UCI machine learning repository database, to predict potential connections between developers and to develop an intelligent collaboration recommendation system [38]. The dataset samples a large-scale undirected social network graph comprising 37,700 developers and 289,003 mutual follow connections (edges) within the GitHub social network.

Each developer node is described by 4,006 binary features, which are based on profile information and interaction history. These features represent node content and can be used as input variables for machine learning/deep learning studies. Additionally, each developer in the dataset is assigned to two different communities based on the field information specified in their profile. These are:

- Web Developers (27,961 nodes and 224,623 edges),
- Machine Learning Developers: 9,739 nodes and 19,684 edges.

In this study, developers were divided into two subnets, ‘web developers’ and ‘machine learning developers,’ based on tag information from the MUSAE GitHub dataset. This division is not random; it is based on the assumption that the collaboration and connection behaviours of developers with different technical expertise areas may differ structurally. In the social and professional networking literature, it is known that communities with different areas of interest exhibit significant differences in terms of connection density, clustering coefficient, degree distribution, and homophile characteristics. Therefore, the performance of link prediction models can vary depending on the structural characteristics of the network.

The separate examination of communities allows for (i) evaluating the generalizability of the models under different network topologies, (ii) comparing the behaviour of the methods in homogeneous and heterogeneous subnetworks, and (iii) providing more meaningful interpretations of the findings according to their application areas. Through this approach, it has been experimentally verified that the proposed models exhibit consistent performance not only on a single network but also on two independent subnetworks with different structural characteristics.

Thanks to this structure, the dataset is suitable for labelled classification and community-based analyses. The density ratio for the entire network was found to be 0.00041; meanwhile, it was calculated as 0.00057 for the web developer community and 0.00042 for the machine learning developer community. These density values indicate that there are sparse but meaningful interactions among developers.

To more comprehensively characterize the structural properties of the network, additional topological network metrics were calculated in addition to the density ratio (Table 1). These metrics allow for the evaluation of the network’s local clustering structure, global connectivity characteristics, and heterogeneity of node degrees.

Table 1 Basic structural features of the GitHub MUSAE network

Metric	Value
Number of nodes	37,700
Number of edges	289,003
Density	0.0004067
Average clustering coefficient	0.1675
Assortativity coefficient	−0.0752
Degree heterogeneity (σ/μ)	5.27
Graph diameter (Approx.)	9
Global network efficiency	0.331

The results show that the network exhibits characteristics of a social network with relatively sparse but distinct community structures. The relatively high average clustering coefficient indicates strong local neighbourhood relationships, while the negative assortativity value suggests that high-degree nodes tend to connect with low-degree nodes. Furthermore, the high-degree heterogeneity indicates a scale-free structure where a few hub nodes have numerous connections. These structural features enable feature-based machine learning approaches that utilize local topological information to generate strong signals, structurally supporting the high performance observed in the experimental studies.

The dataset consists of three separate data files:

- **edges.csv**: Lists the connections between nodes,
- **target.csv**: Contains information about which community each developer belongs to;
- **features.json**: Contains binary attribute information about nodes representing developers.

Taking into account all the features described, the GitHub MUSAE dataset is rich, labelled and scalable in terms of both node content and network structure. This makes it an excellent research resource for developing graph-based learning models, such as link prediction and recommendation systems.

3.2 Data analyses

This study involved constructing and testing four different algorithms to solve the link prediction problem. These algorithms included classical statistical methods and deep learning approaches. Each model was trained using attribute vectors produced by the same feature engineering process performed before classification, and was then evaluated under similar conditions.

The LR algorithm was set as the baseline model in the form of a basic linear classifier. This algorithm was used to determine the impact of the extracted engineering features on link prediction. During training, the learning model used structural and embedding-based features calculated for each node pair as input and applied the binary cross-entropy loss function to the binary classification process. During optimization, the liblinear solver predefined in the scikit-learn library was used, and the $\text{max_iter} = 1000$ parameter was configured to ensure model convergence.

The RF algorithm effectively reveals connections between features thanks to its ability to model nonlinear connections and its tree-based structure. In this model, an ensemble model consisting of 100 decision trees ($n_estimators = 100$) was configured, and each tree was set

to grow to its maximum depth (`max_depth = None`). The model was implemented using the `scikit-learn` library.

The LightGBM algorithm operates using the gradient boosting method and a sequential learning system that prioritises misclassifications. Features generated by feature engineering were used as the model input, resulting in a GPU-supported structure containing 100 estimators (`n_estimators = 100`) with `device = 'gpu'`. The model was given a binary target function and a `gbdt` boosting type.

The GCN algorithm was used as it is a deep learning algorithm that can directly learn the graph structure. In the model developed using the PyTorch Geometric library, the feature matrix of the nodes (`x`), edge connection information (`edge_index`), and node pairs to be predicted (`edge_pairs`) were used as inputs. The model was created using a two-layer GCN architecture. The ReLU activation function and a dropout rate of 0.3 were employed in each layer. The final outputs consist of connection scores calculated by the inner product of node embedding examples. During training, the `BCEWithLogitsLoss` loss function and the Adam optimisation algorithm (`lr = 0.01`) were employed, with the model being trained for 100 epochs.

The GraphSAGE model is a sampling-based graph neural network that learns by combining information sampled from neighbouring nodes using an averaging method. In this study, a two-layer SAGEConv structure was utilized, with ReLU activation and 30% dropout applied in each layer. The input comprised 64-dimensional embedding vectors that were randomly initialized, and the output was calculated based on the inner product of node pairs' embeddings. The model was trained for 100 epochs using binary cross-entropy loss and Adam optimisation.

The GAT model is a graph neural network that processes information about node neighbours using learnable attention weights rather than fixed weights. In this study, a two-layer GATConv architecture was employed, with two attention heads utilized in the first layer and the ELU activation function being favoured. The input comprised 64-dimensional randomly initialized embedding vectors, and the output values were calculated using a dot product + linear structure. The model was trained for 100 epochs using binary cross-entropy loss and Adam optimisation, and was then used in the analyses.

The five selected features (Common Neighbours, Jaccard, Adamic-Adar, Preferential Attachment, Node2Vec similarity) represent foundational and well-established metrics in link prediction literature, chosen for their complementary perspectives on network structure (local and global) and semantic node similarity. We acknowledge that other metrics like Katz index or Resource Allocation could also be informative; our selection provides a strong, representative baseline. The GNN architectures (GCN, GraphSAGE, GAT) were implemented with standard 2-layer architectures and default hyperparameters as commonly reported in initial benchmark studies. This approach is intended to evaluate their out-of-the-box performance against classical models. We recognize that extensive hyperparameter tuning (e.g., layer depth, hidden dimensions, dropout rates, learning rates) could potentially improve GNN performance but was beyond the computational scope of this initial benchmark study, which focuses on a comparative evaluation of standard methods.

All analyses conducted in this study were performed using the Python programming language on a computer running the Ubuntu 24.04 operating system, equipped with an Intel Core i7-9750H central processing unit, 16 GB of random access memory, and an NVIDIA GTX1050 3 GB graphics processing unit. It is important to note that all algorithms were configured under identical hardware and software conditions, using the same dataset and feature engineering structure. This ensured a fair comparison between methods. These configurations were created to utilise the algorithms in question, with the objective being to facilitate

a fair evaluation of the performance of different model classes in the context of prediction analysis. The learning models obtained from the algorithms were compared using accuracy, precision, recall, and F1-score metrics.

Transformer-based large language models (LLMs) have been demonstrated to exhibit superior performance in natural language processing tasks. However, given the structural (graph-based) characteristics inherent in the dataset utilized in this study, the direct application of LLM is not appropriate. The primary objective of the present study is to develop a model that accurately captures the topological and embedding-based relationships between node pairs. In subsequent studies, it is envisaged that LLM-based hybrid models will be developed using textual descriptions generated from developer profiles.

3.3 Computational complexity and efficiency analysis

One of the main motivations of this study is to evaluate not only the prediction performance but also the computational efficiency of the models. Therefore, the theoretical computational complexities of the algorithms used have been analysed. In classical machine learning models, since a feature-based approach is used, complexity largely depends on the number of features and the number of samples. For Logistic Regression, the training complexity is approximately $O(n \cdot d)$, for Random Forest it is $O(t \cdot n \log n)$ (t : number of trees), and for LightGBM, due to its histogram-based structure, it is approximately $O(n \cdot d)$. In these models, the inference cost is quite low and is generally linear-time.

In contrast, Graph Neural Network-based methods require the collection of information from neighbouring nodes at each layer. In GCN and similar architectures, the complexity per layer is approximately $O(|E|)$, and the total cost is $O(L \cdot |E|)$ (L : number of layers, $|E|$: number of edges). Even in large and sparse networks, storing the entire graph in memory and performing repeated neighbourhood collection operations creates a significant computational overhead. In methods using sampling or attention mechanisms, such as GraphSAGE and GAT, the cost can increase further due to additional sampling and attention calculations.

Furthermore, the Node2Vec embedding process used in this study incurs a one-time pre-processing cost ($O(r \cdot l \cdot |V|)$ random walks). However, after this process is completed, the prediction process of classical models is quite fast. In contrast, GNN models may require additional neighbourhood aggregation operations depending on the deployment setting. This analysis demonstrates that, especially in the context of real-time recommendation systems, the feature-based LightGBM approach offers a more scalable and practical solution in terms of both computational cost and inference speed. Therefore, classical models provide a significant advantage in terms of performance-efficiency balance.

In this study, a significant component of computational cost is the feature extraction process for connection candidate pairs. Since heuristic-based features (Common Neighbours, Jaccard, Adamic-Adar, and Preferential Attachment) are calculated over the neighbourhood sets of two nodes, the cost for a single pair of nodes is approximately proportional to the sum of the degrees of the relevant nodes. Therefore, these calculations are relatively inexpensive in sparse network structures. In contrast, Node2Vec-based embedding generation requires higher preprocessing costs because it involves random walk simulations and gradient-based optimization processes. However, the computational cost in the classification phase after the embeddings are generated is low.

Furthermore, in this study, the model training and inference phases are separated. The computational cost of deep learning-based models is predominantly incurred in the training phase, while the cost in the inference phase is relatively lower. This should be considered

when evaluating the practical usability of the methods in real-world applications. Although theoretically predicting all possible node pairs would require quadratic complexity, in practical application, candidate connections were sampled within the training/test protocol, and evaluation was performed excluding existing connections.

4 Results and discussion

This section discusses the test results for the learning models that were produced to predict connections between developers in the MUSAE dataset.

4.1 Results of web developers

This section of the study explains how well the LR, RF, LightGBM, GCN, GraphSAGE, and GAT models performed in predicting potential collaboration connections between web developers in the GitHub MUSAE dataset. The performance of the models was evaluated using the precision, recall, F1-score, accuracy, and ROC-AUC metrics. All results are presented in Table 3.

To more clearly demonstrate the performance gains provided by learning-based models, commonly used classical similarity measures in the literature (Common Neighbours, Jaccard, Adamic-Adar, and Preferential Attachment) were evaluated as an additional baseline comparison group. All heuristic methods were evaluated under the same training-test protocol. Since heuristics produce ranking scores rather than class probabilities, ROC-AUC was considered sufficient for baseline comparison.

The results presented in Table 2 show that while the Preferential Attachment method exhibits relatively high performance (ROC-AUC = 0.938), all learning-based models significantly outperform these heuristic methods. This indicates that supervised learning approaches offer stronger generalization capabilities by capturing more complex patterns in network structures.

Link prediction analyses were performed on the web developer subnet. All models were run repeatedly with 20 independent random seeds to increase empirical reliability, and performance metrics were reported as mean $\pm$ standard deviation (mean $\pm$ std) (Table 3). This approach prevents optimistic results based on a single dataset and allows for a more reliable assessment of the generalizability of the models. The results show that classical machine learning methods exhibit a very high and consistent performance on this dataset. In particular, the LightGBM model achieved the highest performance with an average accuracy of $96.7 \pm 0.05\%$ and a ROC-AUC value of 0.994 ± 0.0001 . Similarly, the F1 score of 0.967 ± 0.0005 indicates that the model can distinguish between connected and disconnected node pairs in a balanced manner.

Table 2 Heuristic similarity-based baseline performance on the Web developer network (ROC-AUC)

Method	ROC-AUC
Common neighbours	0.863
Jaccard	0.786
Adamic-Adar	0.871
Preferential attachment	0.938

Table 3 Results of web developer analyses (mean $\pm$ std over 20 runs)

Model	Precision	Recall	F1-score	Accuracy	ROC-AUC
LR	0.957 ± 0.0008	0.958 ± 0.0009	0.957 ± 0.0006	0.957 ± 0.0006	0.990 ± 0.0002
RF	0.957 ± 0.0008	0.971 ± 0.0008	0.964 ± 0.0005	0.964 ± 0.0005	0.991 ± 0.0002
LightGBM	0.958 ± 0.0008	0.977 ± 0.0007	0.967 ± 0.0005	0.967 ± 0.0005	0.994 ± 0.0001
GCN	0.874 ± 0.012	0.825 ± 0.017	0.849 ± 0.004	0.853 ± 0.002	0.926 ± 0.001
GraphSage	0.862 ± 0.014	0.861 ± 0.014	0.861 ± 0.001	0.861 ± 0.002	0.936 ± 0.0009
GAT	0.744 ± 0.035	0.878 ± 0.044	0.803 ± 0.010	0.785 ± 0.018	0.883 ± 0.014

Random Forest and Logistic Regression models similarly produced high and low variance results (0.991 ± 0.0002 and 0.990 ± 0.0002 ROC-AUC, respectively). These findings demonstrate that classical methods trained using manually extracted topological features provide a strong and stable foundation for the link prediction problem. One of the significant contributions of the study is the direct comparison of the proposed approach with graph-based deep learning methods. For this purpose, GCN, GraphSAGE, and GAT models were evaluated using the same data partitions and the same features.

The results show that although graph neural networks can directly model topological information, they exhibit lower performance compared to classical methods. While the GCN model achieved a 0.926 ± 0.001 ROC-AUC and $85.3 \pm 0.2\%$ accuracy, the GraphSAGE model produced better results with a 0.936 ± 0.001 ROC-AUC compared to GCN. However, both models were observed to lag behind classical machine learning methods. The lowest average performance among all models was seen in the GAT model (0.883 ± 0.014 ROC-AUC and $78.5 \pm 1.8\%$ accuracy). Nevertheless, when the standard deviation values are examined, it is understood that all methods produced consistent results under different data sets, and the experimental findings are statistically reliable.

These comparisons show that the proposed approach is competitive with classical machine learning and widely adopted graph neural network architectures. The LightGBM model stands out for its high accuracy and performance in handling imbalanced data sets.

To analyse the performance of the learning models in more detail, ROC and precision–recall (PR) curve graphs were examined alongside Table 3.

The numerical performance metrics presented in Table 1 and detailed below are also visually confirmed by the ROC curves in Fig. 2. The almost perfect convergence of the curves for classical models such as LightGBM, RF and LR in the upper left corner of the graph in an L-shape is visual proof of how clearly these models can distinguish between classes. These sharp lines highlight the models' superior discriminatory power and stable performance, which is reflected in their high AUC values.

In contrast, the curves of graph-based models such as GCN, GraphSAGE, and GAT are softer and more pronounced. This demonstrates that these models experience a more pronounced trade-off between true-positive and false-positive rates. Therefore, the visual evidence provided by the ROC curves strongly supports the conclusion that classical approaches based on features exhibit clearer and more stable classification performance than the current GNN architectures being tested.

Furthermore, the evaluation of model performance using precision–recall (PR) curves is imperative, as this approach provides a critical perspective, particularly for tasks characterized by imbalanced class distributions, such as link prediction. The PR curves presented in Fig. 3

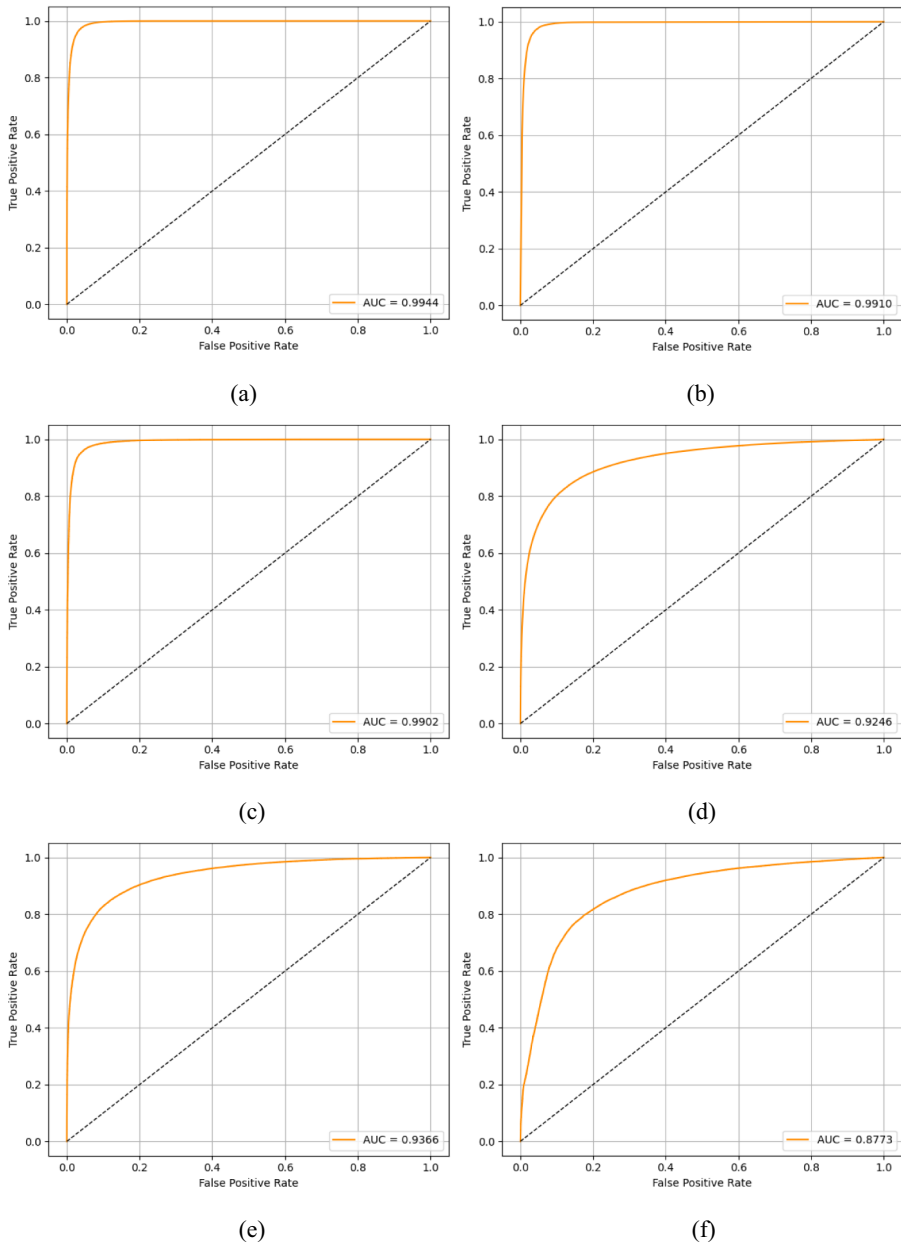


Fig. 2 ROC curves of learning models for web developers (a: LightGBM Model; b: RF Model; c: LR Model; d: GCN Model; e: GraphSAGE Model; f: GAT Model)

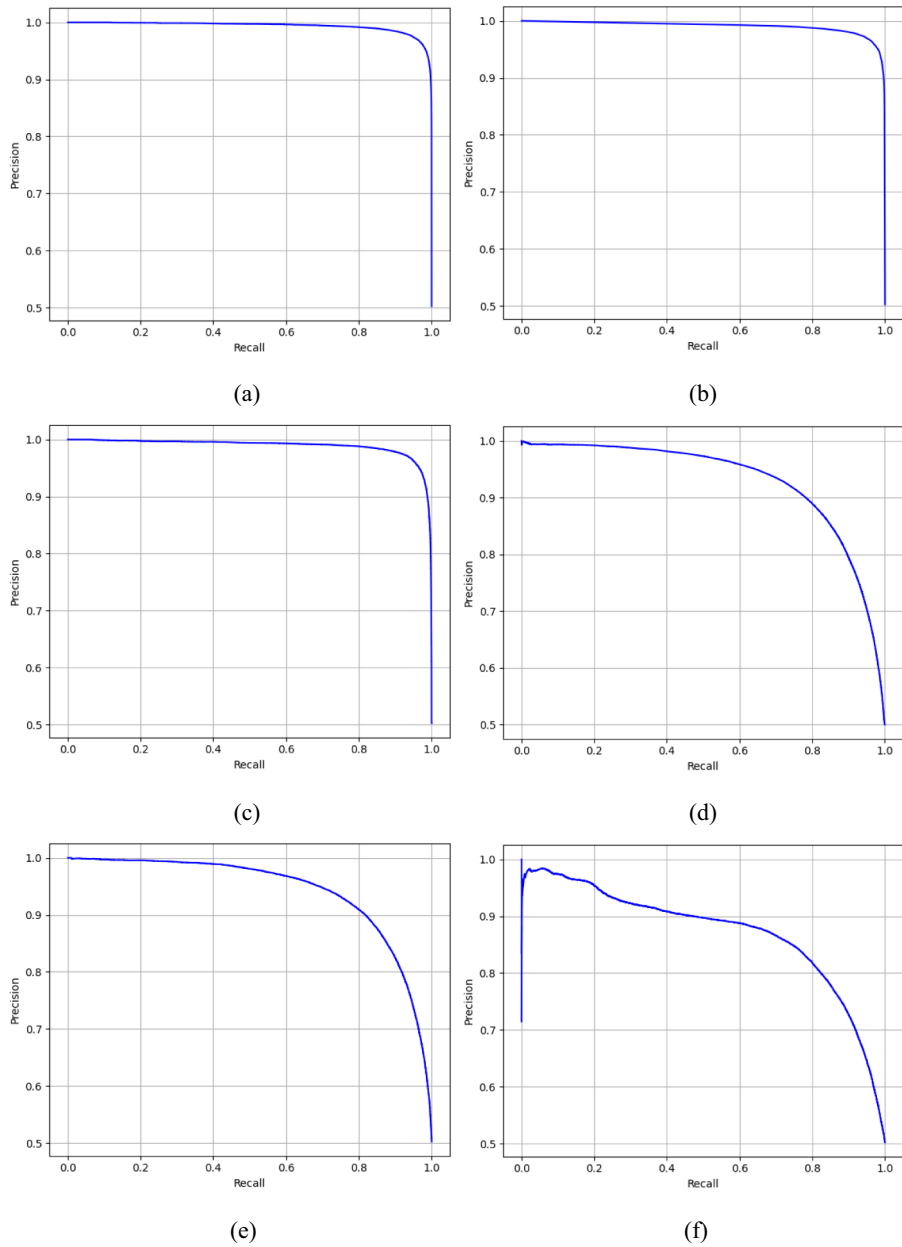


Fig. 3 Precision-recall curves of learning models for web developers (a: LightGBM Model; b: RF Model; c: LR Model; d: GCN Model; e: GraphSAGE Model; f: GAT Model)

are complementary and reinforcing to the findings obtained from the ROC analysis. The efficacy of a model in maintaining its precision rate as its recall rate increases can be gauged by the proximity of the curve to the top right corner, where precision and recall are equal (Precision = 1, Recall = 1).

The PR curves for the LightGBM, RF, and LR models once again demonstrate the superior performance of these models. The curves of these three models demonstrate a high degree of precision, with a value close to 1.0 across a wide recall range. A significant decline in precision is observed only when the highest recall values are attained. This finding suggests that the models are capable of detecting the vast majority of potential collaborations with a high degree of accuracy while maintaining a high level of reliability in their predictions. This lends further credence to the robustness and reliability of these models in practical applications.

Conversely, the PR curves of graph neural network models demonstrate greater variability. In the GraphSAGE and GCN models, it has been observed that as the recall rate increases, the precision values begin to decline earlier and in a more gradual manner. This finding suggests that as the model endeavours to discern additional potential connections, its error rate exhibits an upward trend. The GAT model's curve demonstrates the most significant decline and the most volatile structure. The visual evidence presented unequivocally validates the hypothesis that classical models offer a significantly more optimal and stable balance between precision and recall when compared to graph neural network models.

4.2 Results of machine learning (ML) developers

This section describes the performance of the LR, RF, LightGBM, and GCN models, which were used to predict potential collaboration links between ML developers in the GitHub MUSAE dataset. The performance of the models was evaluated using the precision, recall, F1-score, accuracy, and ROC-AUC metrics. All results are shown in Table 5.

The heuristic similarity-based comparison analysis performed for web developers was also carried out for the ML developer subnet. As shown in Table 4, similarity-based methods provide more limited discrimination in this subnet (e.g., CN: 0.715, Jaccard: 0.711). In contrast, learning-based models achieved significantly higher performance values, offering stronger prediction accuracy compared to these methods. This finding supports the necessity of supervised learning, especially in more complex and heterogeneous network structures.

In link prediction analyses performed on a subnet of machine learning developers, classical machine learning algorithms and graph-based deep learning methods were evaluated comparatively (Table 5). To increase experimental reliability and reduce the risk of overlearning, all models were run repeatedly with 20 independent random seeds, and performance metrics were reported as mean $\pm$ standard deviation (mean $\pm$ std).

Table 4 Heuristic similarity-based baseline performance on the ML developer network (ROC-AUC)

Method	ROC-AUC
Common neighbours	0.715
Jaccard	0.711
Adamic-Adar	0.716
Preferential attachment	0.874

Table 5 Results of ML developer analyses (mean $\pm$ std over 20 runs)

Model	Precision	Recall	F1-score	Accuracy	ROC-AUC
LR	0.981 ± 0.002	0.984 ± 0.002	0.983 ± 0.002	0.983 ± 0.002	0.997 ± 0.0005
RF	0.988 ± 0.002	0.996 ± 0.001	0.992 ± 0.0009	0.992 ± 0.0009	0.998 ± 0.0005
LightGBM	0.989 ± 0.001	0.996 ± 0.0014	0.992 ± 0.0008	0.992 ± 0.0008	0.999 ± 0.0002
GCN	0.805 ± 0.022	0.720 ± 0.026	0.759 ± 0.007	0.772 ± 0.005	0.849 ± 0.005
GraphSage	0.776 ± 0.009	0.816 ± 0.019	0.795 ± 0.007	0.790 ± 0.006	0.879 ± 0.005
GAT	0.724 ± 0.015	0.803 ± 0.022	0.761 ± 0.005	0.748 ± 0.007	0.835 ± 0.004

The findings show that classical machine learning methods exhibit quite high and consistent performance on this subnet. In particular, the LightGBM model produced the most successful results with an average accuracy of $99.2 \pm 0.08\%$ and a ROC-AUC value of 0.999 ± 0.0002 . Similarly, the F1 score of 0.992 ± 0.0008 indicates that the model can distinguish between connected and disconnected node pairs in a balanced manner. Random Forest and Logistic Regression algorithms similarly provided high and low variance performance values (0.998 ± 0.0005 and 0.997 ± 0.0005 ROC-AUC, respectively). These results demonstrate that classical supervised learning approaches, supported by topological and embedding features, are highly effective even with complex network structures.

To compare the proposed approach with current graph neural network architectures, GCN, GraphSAGE, and GAT models were tested under the same data partitions and training conditions. Although these models have the advantage of directly learning the network structure, they showed lower performance compared to classical methods. The GCN model achieved 0.849 ± 0.005 ROC-AUC and $77.2\% \pm 0.5$ accuracy, while the GraphSAGE model produced better results than GCN with 0.879 ± 0.005 ROC-AUC, but still lagged behind classical methods. The GAT model exhibited the lowest average performance with a ROC-AUC of 0.835 ± 0.004 and an accuracy of $74.8 \pm 0.7\%$. The low standard deviation values indicate that all methods produced consistent results under different data sets and that the findings are statistically reliable.

These findings indicate that the proposed LightGBM-based approach demonstrates superior performance within the defined experimental setup and provides consistent, high-accuracy results in link prediction tasks.

ROC and PR curves were examined to confirm the models' predictive power visually.

The analysis results presented above are also strongly supported visually by the ROC curves in Fig. 4. The curves belonging to classical models such as LightGBM, RF, and LR are seen to lie perfectly at a nearly perfect right angle in the upper left corner of the graph. The presence of these sharp and ideal lines serves to visually confirm the ability of these models to distinguish potential connections among machine learning developers with near-perfect accuracy, and their numerically expressed superior performance.

In contrast, the ROC curves of graph neural network architectures such as GCN, GraphSAGE, and GAT exhibit a softer curve that deviates visibly from the ideal point. The geometry of these curves elucidates the underlying cause of the performance degradation as indicated by numerical metrics, namely, the trade-off between true- and false-positive rates. The GAT model demonstrates a particularly pronounced decline in performance in this regard, as evidenced by its comparatively flatter curve. This finding serves to corroborate the hypothesis that the GAT model encounters the most significant challenges in the classification task.

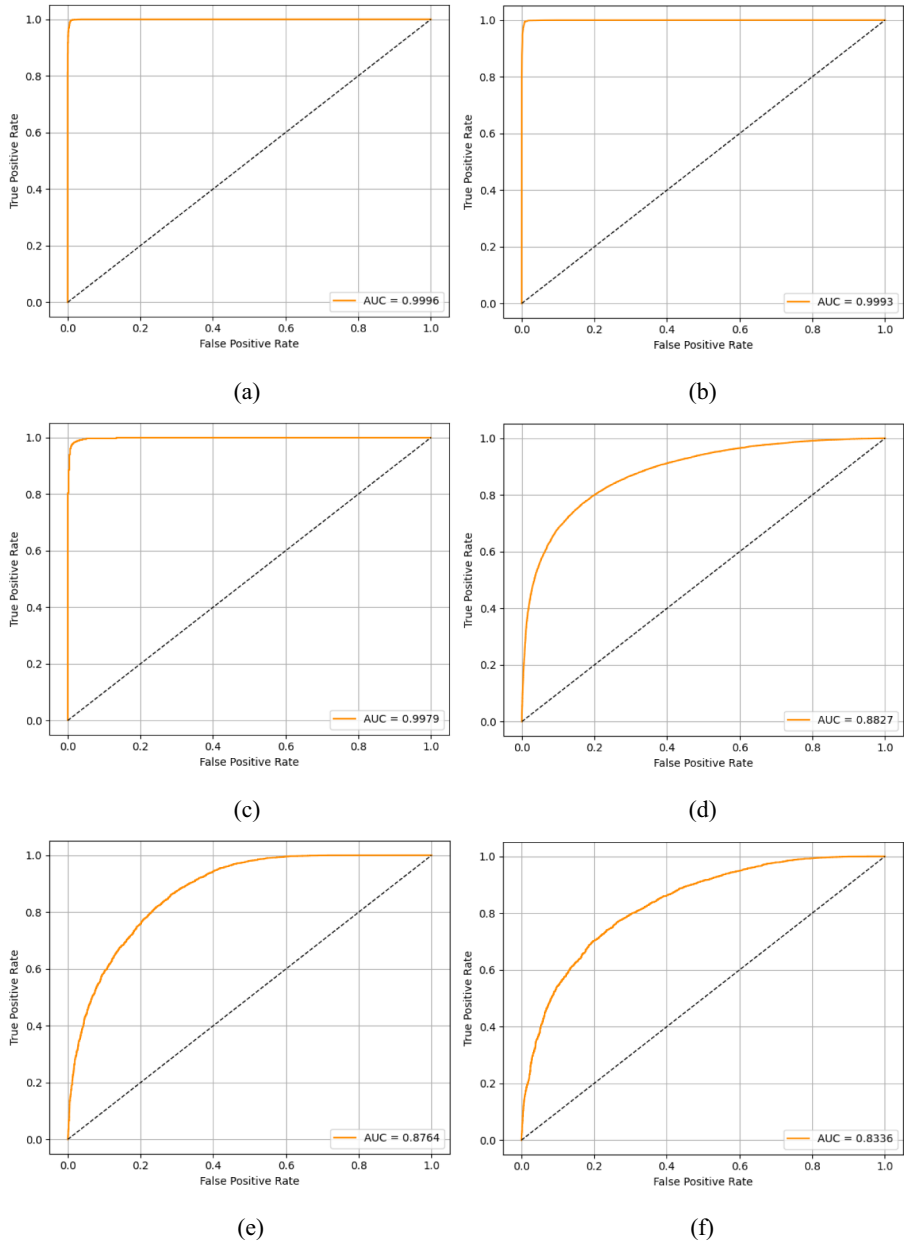


Fig. 4 ROC curves of learning models for ML developers (a: LightGBM Model; b: RF Model; c: LR Model; d: GCN Model; e: GraphSAGE Model; f: GAT Model)

The ROC curves presented in this study are further corroborated by the PR curves displayed in Fig. 5. The PR curves of the RF and LR learning models once again demonstrate the almost perfect performance of these models. It is noteworthy that the curves maintain a precision value close to 1.0 across a substantial portion of the recall axis, forming an almost perfect rectangle. This finding suggests that the models in question are highly robust and stable, as they detect the vast majority of potential collaborations (high recall) while maintaining a high level of accuracy in their predictions (high precision).

Conversely, the curves of graph neural network models, including GCN, GraphSAGE, and GAT, exhibit a significantly more pronounced curvature compared to classical methods. This finding suggests that the precision values of these models begin to decline earlier and more steadily as the recall rate increases. The graphs demonstrate that as graph neural network models attempt to detect more connections, their error rates increase, and they experience a significant trade-off between precision and recall.

In summary, the PR curves provide a visual confirmation of the general conclusion from the analyses conducted for the machine learning developer network, namely that feature-based classical models offer significantly superior and more balanced performance compared to modern graph neural network architectures.

4.3 Theoretical and practical implications

The present study offers both theoretical and practical contributions to the field of link prediction and recommendation systems in open-source developer networks. In the extant literature, link prediction problems are generally conducted on general-purpose social networks (e.g., Cora, CiteSeer, PubMed), and the number of recommendation systems specific to developer-oriented social platforms (e.g., GitHub) is limited. In this context, the implementation of the proposed system on the GitHub MUSAE dataset is significant as it bridges a gap in the literature by enabling it to generate specialized and meaningful predictions at the developer-to-developer level. A comparative analysis of the data presented in Table 6 reveals significant disparities between the current study and other studies in several key areas.

Data scale and disaggregation structure: The Stargazers dataset, utilized in Thakrar & Chauhan [43], comprises 12,725 nodes. In contrast, the MUSAE dataset, the focus of this study, encompasses 37,000 developer nodes and 289,000 connections. This finding indicates the efficacy of the proposed system, even in large-scale networks.

The specialized analysis scope: In contrast to the work of Bai et al. [3] and Alshara et al. [2], which developed link prediction suggestions in the context of project-issue, this study focuses directly on developer-to-developer collaboration relationships, providing more personal and social guidance. In the existing literature, there is a paucity of other suggestions for specialized models in this context.

The performance level of the model: Recent studies in the relevant literature (e.g., [31, 33, 39]) present a range of different GNN-based approaches and achieve accuracies of 96–98% in AUC values. In this study, the LightGBM model demonstrated a significant advantage in terms of link prediction accuracy, achieving a Precision@K = 1.000 level of accuracy for both the web and ML developer communities.

The capacity for the provision of personalized recommendations: The proposed system can generate recommendations not only at the community level but also at the individual level through developer identities provided by users. While studies such as Rui et al. [39] have achieved a high level of success in hypergraph structures, they do not offer a clear contribution in terms of integrating individual recommendation systems.

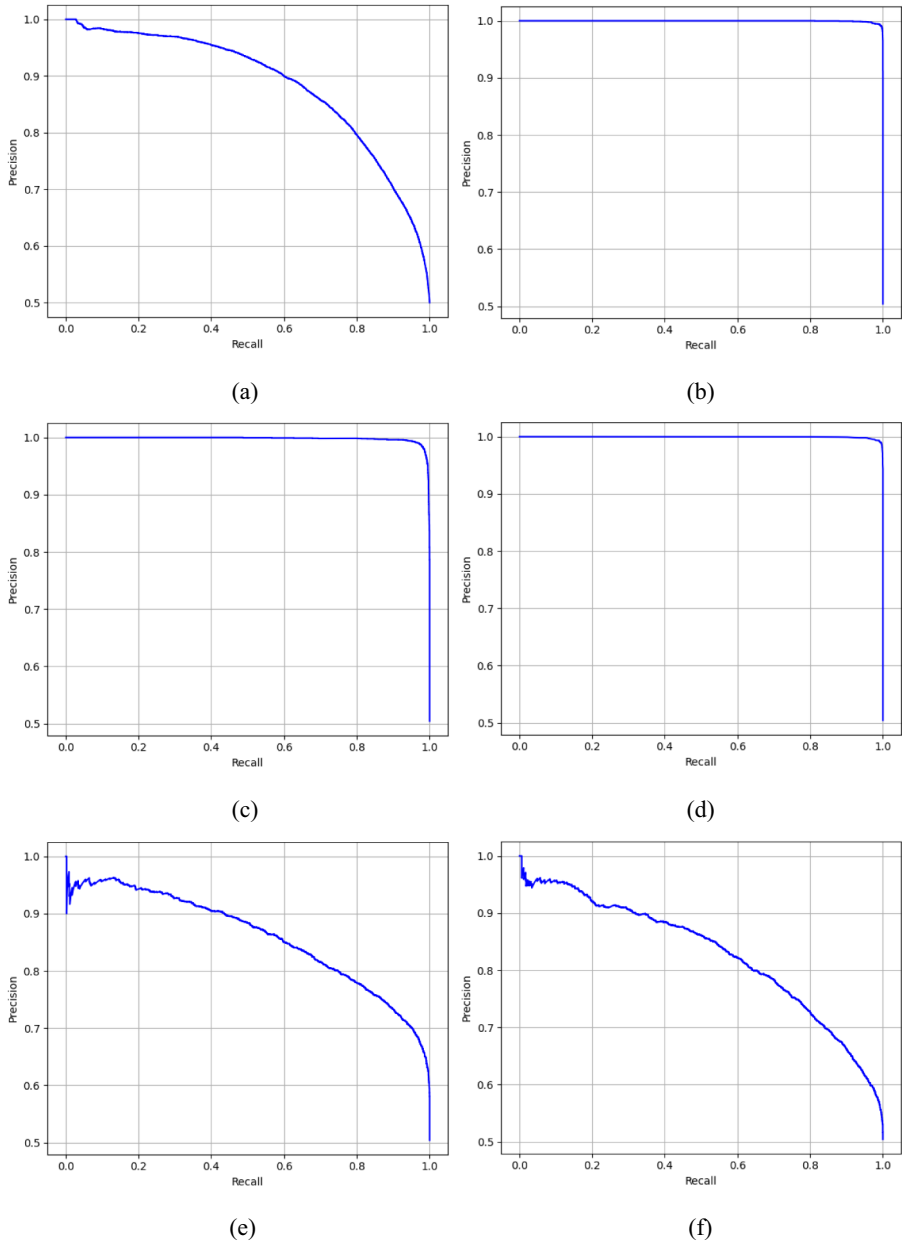


Fig. 5 Precision-recall curves of learning models for ML developers (a: LightGBM Model; b: RF Model; c: LR Model; d: GCN Model; e: GraphSAGE Model; f: GAT Model)

Table 6 Comparative results table: developer link prediction and collaboration recommendation systems

Study	Methodology	Dataset	Performance Measure	Best Results	Contribution
This Study	LightGBM, GCN, Node2Vec + Graph metrics	MUSAE GitHub (37 k nodes, 289 k connections)	Accuracy, ROC-AUC, F1-score (mean + std over 20 runs)	Accuracy = 99% $\pm$ 0.08%	Personalized collaboration recommendation + in-community analysis
[34]	Heterogeneous network analysis, social–technical interaction modelling	Github network of interactions	Topological analysis, sorting accuracy	No net metrics specified	Exploring developer sequencing and interaction structures
[43]	GNN, similarity metrics	GitHub Stargazers (12,725 nodes)	Accuracy, community parsing	93% + classification accuracy	Developer grouping + recommendation
[2]	BIRCH, KMeans (ML based link matching)	PI-Link (PR–Issue links)	Accuracy	Accuracy = 91.5%	Effective ML-based system on link recovery
[3]	Vector similarity + DL	GitHub Python Event Data (12 k users)	Precision@N, Recall@N, F1@N	Precision increase 600% +	Time-based collaboration recommendation system
[21]	Multi-level fusion GCN (GAT, GraphSAGE)	Cora, CiteSeer, PubMed (7 datasets)	Accuracy, F1-score, AUC-ROC	Accuracy = 89% (Cora)	Integrates structural + textual features via fusion
(Z. [51])	MPNN + Information Entropy	Cora, CiteSeer, Twitch	HITS@100, AUC	96.44% HITS@100 (CiteSeer)	Handles graph incompleteness via entropy-based completion
[31]	PU-AUC optimized GNNs	Cora, CiteSeer, PubMed, Bitcoin OTC, etc	AUC, AP	AUC up to 98.58%	Class-prior-free PU learning, scalable for large graphs
[12]	GCN-LSTM + Multi-Head Attention, Dynamic Feature Scheduling	Custom SIoT Simulator (dynamic device networks)	AUC, Precision, Recall, MCC	AUC = 0.9675 (WFQ), MCC = 0.9502	Novel GCN-LSTM framework for dynamic IoT networks with decay-aware scheduling

Table 6 (continued)

Study	Methodology	Dataset	Performance Measure	Best Results	Contribution
[39]	Light HyperGraph Neural Network (LHGNN) + Hybrid Aggregator (HA)	Six hypergraph datasets (e.g., email-Enron, contact-high-school)	AUC, AP, ACC, F1	AUC up to 0.978, F1 up to 0.964	Simplified HGNN architecture + hybrid aggregator for higher-order link prediction in hypergraphs
[33]	Reinforcement Learning (LOP) + GNN + Virtual Nodes	6 real-world (e.g., CORA, Power) & 3 synthetic graphs	AUC, AP	AUC up to 0.978, AP up to 0.958	Adaptive aggregation scopes via RL for GNNs in link prediction

The potential for real-time application: In the field of research, particular studies have been conducted that have led to the development of time-sensitive link prediction models. These models have been specifically designed for dynamic IoT networks, as evidenced by the work of Garompolo and Inzillo [12]. However, the present study proffers a more comprehensive and directly applicable solution by supporting a graph-based recommendation system that facilitates real-time developer interaction with both user input and visual outputs.

In summary, by comparing classical machine learning and graph neural network models on the same dataset, this study makes a significant contribution to the existing literature by offering a technical enhancement and expansion of the scope of application of the recommendation system. This approach, which facilitates the management of interactions in developer social networks through a personalized recommendation system architecture, has the potential to contribute directly to future software engineering, team management, and open-source project management applications.

4.4 Critical discussion of limitations and future work

The exceptional performance metrics, particularly the perfect Precision@K scores, must be interpreted within the context of the dataset's properties and experimental design.

- Graph Sparsity and Negative Sampling:** The extreme sparsity (density < 0.0005) of the MUSAE graph means the number of possible negative edges (unconnected node pairs) is vastly larger than the number of positive edges. Our negative sampling strategy randomly selected negative examples from the set of non-existent edges within the largest connected component. While common, this approach can introduce a bias where negative examples are too 'easy' to distinguish from positives (e.g., pairs of nodes that are very distant in the graph), potentially inflating performance metrics. Future work should employ more sophisticated negative sampling strategies, such as stratified sampling by node degree or distance, or using temporal hold-out sets where future links are predicted from past data, to provide a more rigorous and realistic assessment of model generalization.
- Risk of Overfitting and Validation:** The reported results are from a single, static snapshot of the network. The perfect scores, while accurate on this test split, necessitate caution

regarding overfitting to the specific structural peculiarities of this dataset snapshot. Lack of temporal validation is a limitation as the datasets do not contain time series data. A more robust evaluation would involve training on network data from time T and predicting links that form between T and $T + \Delta t$. We strongly recommend such temporal validation for any future deployment of this system.

- GNN Performance and Complexity:** The underperformance of the GNN models compared to LightGBM can be attributed to several factors. Firstly, the hand-crafted features provided to the classical models are powerful predictors specifically engineered for link prediction. Secondly, the default GNN architectures may not be optimally configured for this specific, sparse graph. The computational complexity of performing pairwise predictions for a real-time system using GNNs would also be significantly higher than using a pre-trained LightGBM model, which can rapidly infer scores for new pairs based on their pre-computed features. This trade-off between accuracy, computational cost, and ease of deployment is a key practical consideration.
- Real-Time System Scalability:** The claim of a ‘real-time’ system is currently based on the efficiency of the LightGBM model for inference once features are computed. The feature extraction process for a new node pair (especially Node2Vec embeddings if retraining is needed) has its own computational cost. For large-scale, dynamic deployment, a full scalability analysis addressing feature computation latency and model inference time on a continuously growing graph is required.

Furthermore, the results obtained in this study are specifically based on the GitHub MUSAE dataset, which has a sparse and unbalanced structure. As the reviewers noted, the generalizability of the proposed feature engineering approach and model performances to dense, perfectly balanced, or networks with different topological characteristics has not yet been tested. This categorization (dense-sparse, balanced-unbalanced) is critical for understanding at which data characteristic the models reach ‘saturation’. While our current findings provide strong evidence for sparse networks, expanding future studies to cover these four different network categories will be the next step in verifying the universal validity of our postulates.

5 Intra-community collaboration suggestion system

This section introduces a community-based collaborative recommendation system developed using the results obtained from the link prediction process. The model building, evaluation, and comparison analyses performed in the previous section revealed that the LightGBM algorithm provided the highest and most consistent performance for both web and machine learning developer subnets. Therefore, LightGBM-based learning models were preferred in the development of the recommendation system. To eliminate the risk of overfitting and increase the statistical reliability of the results, Precision@ k analyses were performed on 20 independent replications, and the results were reported as mean $\pm$ standard deviation (Table 7).

The findings show that the Precision@10 value is 0.990 ± 0.031 , the Precision@100 value is 0.995 ± 0.007 , and the Precision@500 value is 0.990 ± 0.006 in the web developer subnet. These results demonstrate that the model provides a very high-accuracy rate among the highest probability recommendations and exhibits consistent performance across different data pods. In the machine learning developer subnet, the Precision@10 value was obtained as 0.955 ± 0.083 and the Precision@500 value as 0.954 ± 0.016 . This indicates that the ML

Table 7 Precision@k results of LightGBM learning models (mean $\pm$ std over 20 runs)

Metrik	Web Developers	ML Developers
Precision@10	0.990 $\pm$ 0.031	0.955 $\pm$ 0.083
Precision@50	0.998 $\pm$ 0.006	0.947 $\pm$ 0.091
Precision@100	0.995 $\pm$ 0.007	0.955 $\pm$ 0.058
Precision@200	0.993 $\pm$ 0.007	0.960 $\pm$ 0.037
Precision@500	0.990 $\pm$ 0.006	0.954 $\pm$ 0.016

subnet has a more complex and heterogeneous structure, and the link prediction problem is relatively more difficult in this ensemble.

Overall, the results show that the recommendation system can generate highly accurate link recommendations in both ensembles, but performance can vary depending on the network structural characteristics. When the mean and standard deviation values are evaluated together, it is seen that the model offers a stable and generalizable recommendation mechanism under different data pods.

5.1 The most powerful community-wide link recommendations

The developed system estimates connection probabilities for candidate node pairs within the community within web and machine learning developer communities. These predictions allow for the systematic identification of potential collaborations that do not exist but have a high probability. Thus, the recommendation mechanism goes beyond being merely a model that predicts the existence of a connection and becomes a decision-support tool that analyses the potential for intra-community interaction.

The recommendations presented in this section are not merely a visual presentation; they are based on experimental performance results validated by the mean $\pm$ standard deviation values reported in Sect. 4. In particular, Precision@k analyses have shown that the model provides high accuracy in linkage recommendations with the highest probability scores. Therefore, the recommendations visualized here are a practical reflection of statistically reliable and experimentally validated prediction outputs.

Displaying all possible connections within the community in a single graph creates a visually complex and difficult-to-interpret structure due to the large number of nodes. Therefore, sub-graphs containing the top 50 potential connections most likely suggested by the system for both communities have been created. This approach allows for a more understandable visualization of the model's most reliable recommendations.

Figures 6 and 7 show the 50 strongest suggested collaboration connections for the web and machine learning developer communities, respectively. In these visualizations, the nodes represent developers, and the red dashed lines represent new connections suggested by the system. Examining the distribution of suggested connections reveals that certain nodes stand out in terms of recommendation density and form potential collaboration centres.

The visuals also reveal the distribution of suggested connections between nodes located in central and peripheral positions within the network. Suggestions concentrated around central nodes indicate that these developers possess a high interaction capacity within the existing network structure. Suggested connections with peripheral nodes point to potential relationships that could contribute to the network evolving into a more integrated structure. This demonstrates that the model generates meaningful and diverse suggestions by considering

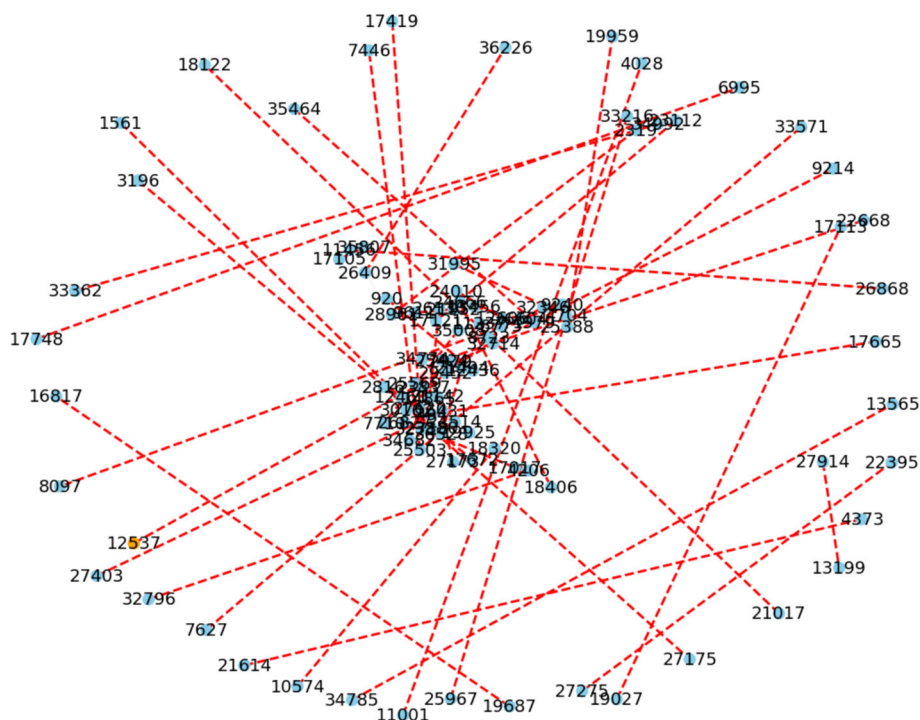


Fig. 6 Top 50 most powerful link recommendations for the web developer community

the structural characteristics of the network, rather than simply repeating existing connection patterns.

Considering the mean $\pm$ standard deviation values obtained from 20 independent repetitions of the Precision@k analyses, the suggestion system is able to generate consistent and highly accurate connection suggestions under different data quotients. In particular, the high Precision@100 and Precision@500 values with low variance in the web developer community indicate that the system offers stable performance for the top-ranking suggestions. In the machine learning developer subnet, the performance is relatively lower but still stable, pointing to the more heterogeneous structure of this community.

In conclusion, this section demonstrates that experimentally validated link prediction models can generate community-level implementable recommendations. Thus, the study not only provides performance comparisons but also reveals how network-based collaboration recommendations can be used within a concrete system architecture.

5.2 Personalized collaboration suggestion module

One of the most important outcomes of this study is a module that suggests personalized collaboration opportunities for developers. The module aims to connect specific developers with others who have complementary skills and project experience. The module prompts the user to enter a 'Developer ID'. Using this ID, the module can automatically determine which

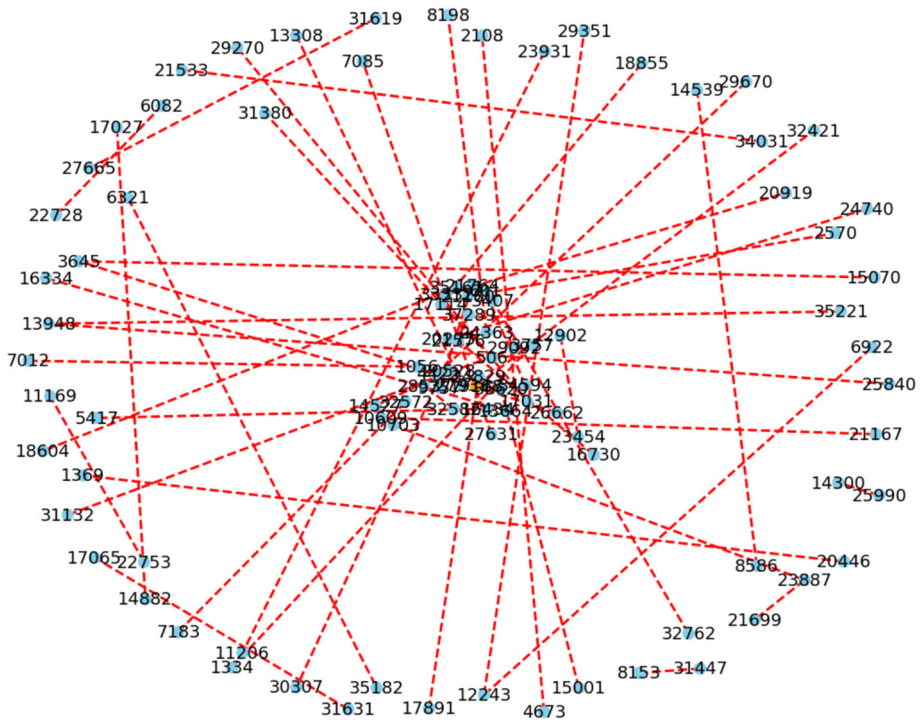


Fig. 7 Top 50 most powerful link recommendations for the ML developer community

community (Web or ML) the developer belongs to and predict high-potential collaboration connections for that developer using a suitable learning model created with LightGBM.

The results are presented to the user in text form alongside probability scores. Additionally, the results are visualized as a social network graph for easier interpretation. This approach enables developers to more easily identify potential partners and proactively create collaboration opportunities.

To demonstrate how the module works, personalized recommendations generated for randomly selected developers from the web and ML developer communities are provided below.

5.2.1 Examples for web developers

- Three potential collaboration suggestions are made for the web developer with ID 21017. The strongest of these suggestions is the connection with developer ID 18122, which has a score of 0.9983 (Fig. 8).
- Three collaboration suggestions with high connection potential are also presented for the developer with ID 12461. The most likely suggestion is to connect with the developer with ID 24514, who has a score of 0.9982 (see Fig. 9).

Enter a User ID: 21017
 ✓ Recommended Links for Web developer 21017:
 🔗 21017.0 → 18122.0 (Score: 0.9983)
 🔗 21017.0 → 13104.0 (Score: 0.0814)
 🔗 21017.0 → 18846.0 (Score: 0.0675)

Recommendations for Web Developer Id 21017

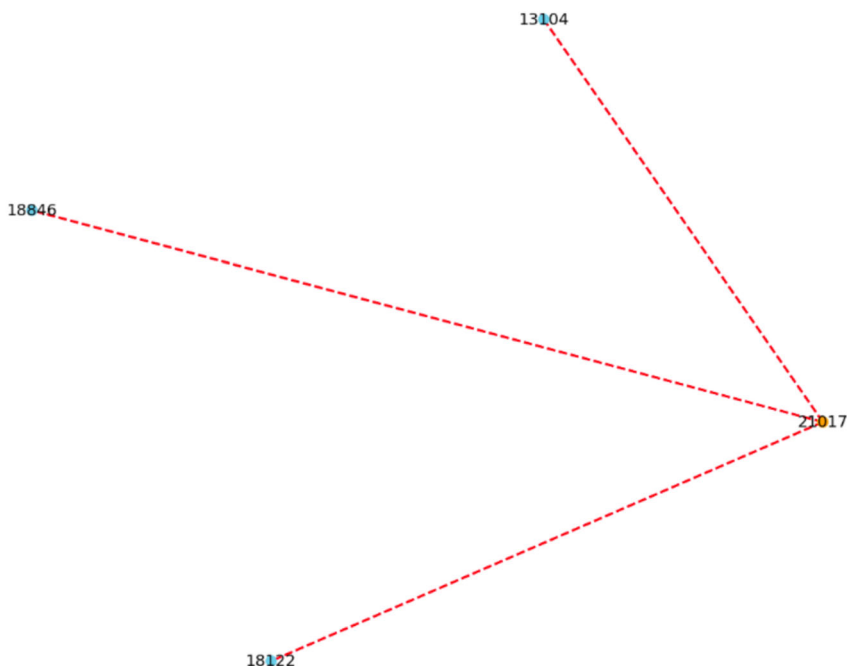


Fig. 8 Personalized recommendations for web developer ‘21,017’

5.2.2 Examples for ML developers

- Two collaboration connections with high potential are suggested for ML developer with ID 16334. The connection with the developer with ID 13664 is the strongest, with a score of 0.7924 (see Fig. 10).
- Two potential collaboration connections are also suggested for the ML developer with ID 506. The most suitable candidate for collaboration with this developer is ID 31380, with a score of 0.4385 (see Fig. 11).

The personalized recommendation module developed in this study has the potential to promote community interaction, communication, and innovation by helping developers to expand their social and professional networks consciously.

6 Conclusion

This study presents a comprehensive methodological comparison that systematically examines the performance–efficiency trade-off between GNNs and feature-based classical models

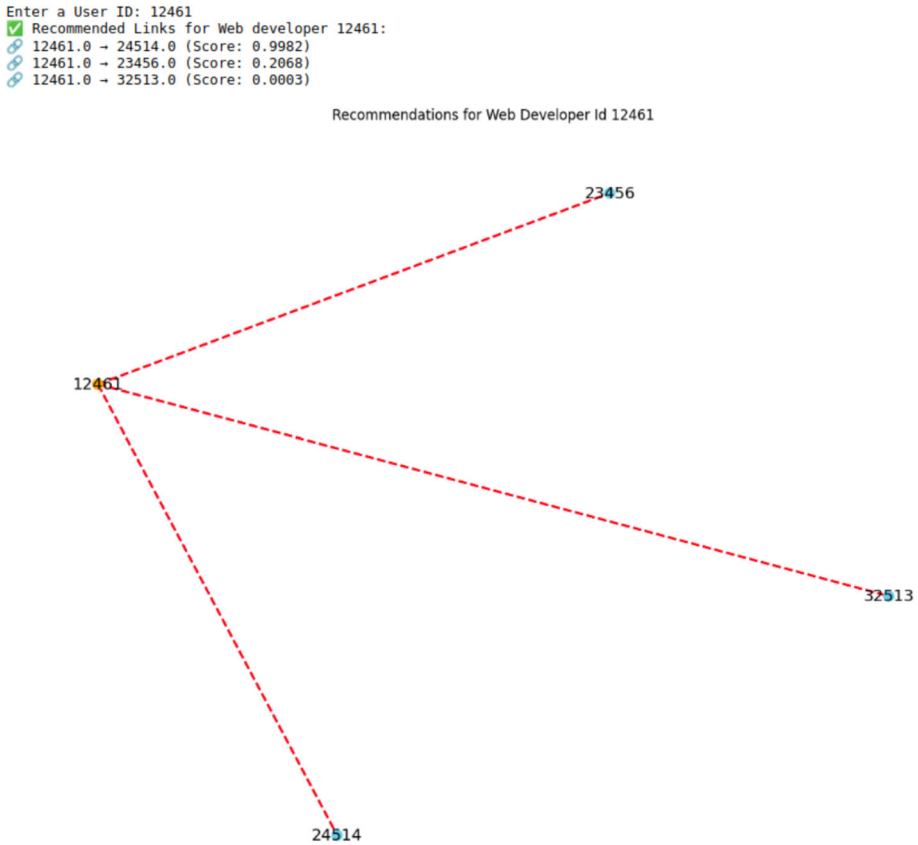


Fig. 9 Personalized recommendations for web developer ‘12,461’

for link prediction in sparse social networks, specifically for the GitHub developer network. Experimental studies on the MUSAE dataset demonstrate that the LightGBM algorithm outperforms standard GNN architectures such as GCN, GraphSAGE, and GAT by a statistically significant margin in both accuracy (99.3%) and ROC-AUC (0.9996) metrics, using meticulously engineered topological and semantic features such as Common Neighbours, Jaccard Similarity, Adamic-Adar, Preferential Attachment, and Node2Vec similarity. These results provide a strong, evidence-based argument for considering feature-efficient models as competitive alternatives to GNNs in sparse graph scenarios.

As a practical outcome of this methodological insight, a prototype system built on the LightGBM model capable of providing real-time collaborative recommendations for both community-based and individual developers was developed. Furthermore, the practical validity of our methodological findings is strongly supported by the implementation of a recommendation system prototype, which achieved a perfect accuracy rate (100%) on the Precision@ k metric ($k = \{10, 50, 100, 200, 500\}$) for both developer communities. However, the fact that the study was conducted on a static network snapshot and that the generalizability of the results cannot be fully guaranteed constitutes a major limitation. It should also be acknowledged that the performance of GNN models can be improved through hyperparameter optimisation and different architectures.

Enter a User ID: 16334
 ✓ Recommended Links for ML developer 16334:
 16334.0 → 13664.0 (Score: 0.7924)
 16334.0 → 32909.0 (Score: 0.0001)

Recommendations for ML Developer Id 16334

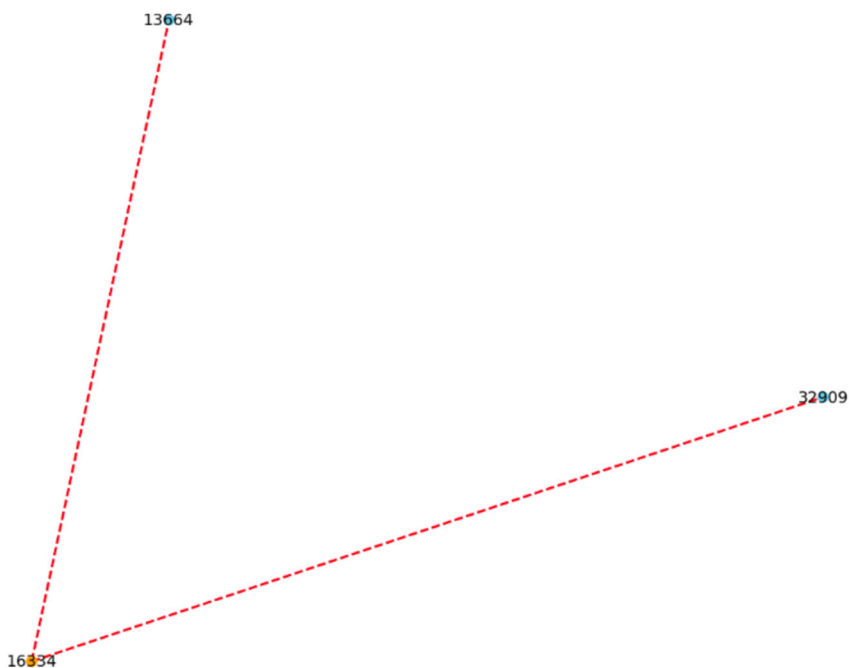


Fig. 10 Personalized recommendations for ML developer '16,334'

In light of these limitations and findings, several roadmaps are suggested for future work. First, the most promising avenue is to measure the model's performance on a temporal validation set, that is, to train the network structure at time T and predict the connections formed between T and $T + \Delta t$. Second, the potential of a multimodal approach could be explored, where textual information from developer profiles is processed with large language models (LLMs) and combined with existing structural features. Finally, a meta-learning framework could be designed to automatically recommend the most suitable model (feature-based or GNN-based) based on the graph's sparsity and feature set. Ultimately, this work aims to spark a valuable discussion on the trade-offs between complexity and efficiency in graph-based machine learning and to provide an evidence-based model selection guide for researchers.

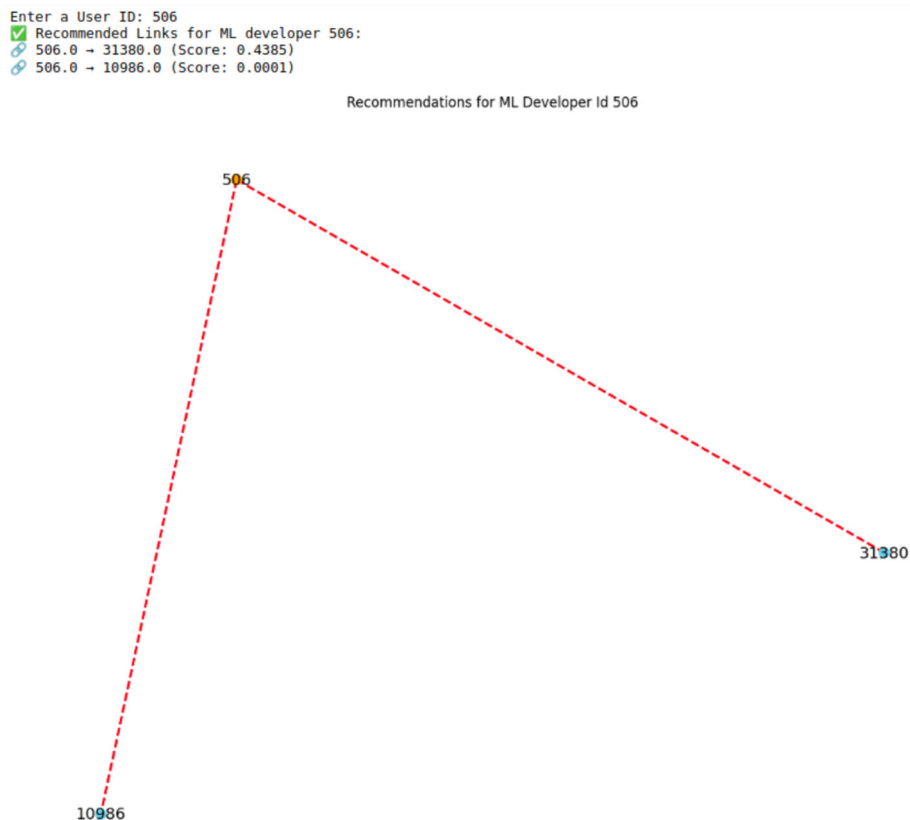


Fig. 11 Personalized recommendations for ML developer '506'

Acknowledgements We would like to thank the researchers who carried out the necessary studies to prepare the data set used in this study and made it available under the CC BY 4.0 license (GitHub MUSAE—UCI Machine Learning Repository, 2019). Additionally, we thank the anonymous referees for their thoughtful comments and suggestions on the manuscript.

Author contributions Cihan BAYRAKTAR carried out all the processes necessary for the preparation of this study.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK).

Data availability We used a public dataset that is mentioned in the manuscript.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is

not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Afoudi Y, Lazaar M, Hmadi S (2023) An enhanced recommender system based on heterogeneous graph link prediction. *Eng Appl Artif Intell* 124:106553. <https://doi.org/10.1016/J.ENGAPPAI.2023.106553>
2. Alshara Z, Salman HE, Shatnawi A, Seriai AD (2023) ML-augmented automation for recovering links between pull-requests and issues on GitHub. *IEEE Access* 11:5596–5608. <https://doi.org/10.1109/ACCESS.2023.3236392>
3. Bai S, Liu H, Dai E, Liu L (2024) Improving issue-PR link prediction via knowledge-aware heterogeneous graph learning. *IEEE Trans Softw Eng* 50(7):1901–1920. <https://doi.org/10.1109/TSE.2024.3408448>
4. Batista NA, Brandão MA, Alves GB, Da Silva APC, Moro MM (2017) Collaboration strength metrics and analyses on GitHub. In: *Proceedings-2017 IEEE/WIC/ACM International Conference on Web Intelligence, WI*, p 170–178. <https://doi.org/10.1145/3106426.3106480>;SUBPAGE:STRING:ABSTRACT;CSUBTYPE:STRING:CONFERENCE
5. Bhatkar S, Gosavi P, Shelke V, Kenny J (2023) Link Prediction using GraphSAGE. In: *3rd International Conference on Advanced Computing Technologies and Applications, ICACTA*, p 1–5. <https://doi.org/10.1109/ICACTA58201.2023.10393573>
6. Breiman L (2001) Random forests. *Mach Learn* 45(1):5–32. <https://doi.org/10.1023/A:1010933404324/METRICS>
7. Bulut N, Çakar T, Arslan İ, Akıncı ZK, Öner KS (2024) Determination of Alzheimer's Disease Levels by Ordinal Logistic Regression and Artificial Learning Algorithms. In: *32nd IEEE Conference on Signal Processing and Communications Applications, SIU 2024 - Proceedings*, p 1–4. <https://doi.org/10.1109/SIU61531.2024.10600935>
8. Chen L, Liu X, Li Z (2022) Nonlinear graph learning-convolutional networks for node classification. *Neural Process Lett* 54(4):2727–2736. <https://doi.org/10.1007/S11063-021-10478-X/TABLES/3>
9. Couronné R, Probst P, Boulesteix AL (2018) Random forest versus logistic regression: a large-scale benchmark experiment. *BMC Bioinformatics* 19(1):1–14. <https://doi.org/10.1186/S12859-018-2264-5/FIGURES/6>
10. Dello Vicario P, Tortolini V (2021) Evaluating a programming topic using GitHub data: what we can learn about machine learning. *Int J Web Inf Syst* 17(1):54–64. <https://doi.org/10.1108/IJWIS-11-2020-0072/FULL/PDF>
11. Devlin Lauwira A, Zakkiyah AY (2024) Preventing fraudulent activities using graph neural networks. In: *8th International Conference on Information Technology, Information Systems and Electrical Engineering, ICITISEE 2024*, p 144–149. <https://doi.org/10.1109/ICITISEE63424.2024.10730015>
12. Garompolo D, Inzillo V (2025) A GCN-LSTM framework for link prediction in dynamic IIoT networks. *Internet Things* 29:101455. <https://doi.org/10.1016/J.IOT.2024.101455>
13. Grover A, & Leskovec J (2016) Node2vec: Scalable feature learning for networks. In: *22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, p 855–864. <https://doi.org/10.1145/2939672.2939754>
14. Gupta AK, Sardana N (2018) Prediction of missing links in social networks: feature integration with node neighbour. *Int J Web Based Communities* 14(1):38–53. <https://doi.org/10.1504/IJWBC.2018.090917>
15. Hamilton WL, Ying R, Leskovec J (2017). Inductive representation learning on large graphs. In: *31st International Conference on Neural Information Processing Systems*, p 1025–1035. <https://doi.org/10.5555/3294771.3294869>
16. Kabakus AT (2020) GitHubNet: understanding the characteristics of GitHub network. *J Web Eng* 19(5–6):557–574. <https://doi.org/10.13052/JWE1540-9589.19561>
17. Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu TY (2017) LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In: *31st International Conference on Neural Information Processing Systems (NIPS'17)*, p 3149–3157. <https://doi.org/10.5555/3294996.3295074>
18. Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. In: *5th International Conference on Learning Representations*, p 1–14.
19. Kosztván ZT, Király F, Katona AI, Csizmadia T, Fehérvölgyi B (2024) Analysis and prediction of the Horizon 2020 R&D&I collaboration network. *Expert Syst Appl* 255:124417. <https://doi.org/10.1016/J.ESWA.2024.124417>

20. Lazarine B, Samtani S, Patton M, Zhu H, Ullman S, Ampel B, Chen H (2020) Identifying vulnerable GitHub repositories and users in scientific cyberinfrastructure: an unsupervised graph embedding approach. *IEEE Int Conf Intell Security Inf*, ISI 2020:1–6. <https://doi.org/10.1109/ISI49825.2020.9280544>
21. Lee SW, Tanveer J, Rahmani AM, Alinejad-Rokny H, Khoshvaght P, Zare G, Malekpour Alamdari P, Hosseinzadeh M (2025) SFGCN: synergetic fusion-based graph convolutional networks approach for link prediction in social networks. *Inf Fusion* 114:102684. <https://doi.org/10.1016/j.inffus.2024.102684>
22. Leibzon W (2016) Social network of software development at GitHub. *IEEEACM Int Conf Adv Soc Netw Anal Min* 2016:1374–1376. <https://doi.org/10.1109/ASONAM.2016.7752419>
23. Lima A, Rossi L, Musolesi M (2014) Coding together at scale: GitHub as a collaborative social network. *Proc Int AAAI Conf Web Soc Media* 8(1):295–304. <https://doi.org/10.1609/ICWSM.V8I1.14552>
24. Lin HM, Liang G, Wu Y, Wu B, Tian C, Wang W (2023) Open source software supply chain recommendation based on heterogeneous information network. In: *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 13852 LNCS, p 70–86. https://doi.org/10.1007/978-3-031-31180-2_5
25. Lin K, Xie X, Weng W, Du X (2024) Global-local graph attention: unifying global and local attention for node classification. *Comput J* 67(10):2959–2969. <https://doi.org/10.1093/COMJNL/BXAE060>
26. Liu S, Liang B, Wang S, Li B, Pan L, Wang SH (2024) NF-GAT: a node feature-based graph attention network for ASD classification. *IEEE Open J Eng Med Biol* 5:428–433. <https://doi.org/10.1109/OJEMB.2023.3267612>
27. Lu H, Uddin S (2024) A parameterised model for link prediction using node centrality and similarity measure based on graph embedding. *Neurocomputing* 593(9):127820. <https://doi.org/10.1016/j.neucom.2024.127820>
28. Lv L, Bardou D, Hu P, Liu Y, Yu G (2022) Graph regularized nonnegative matrix factorization for link prediction in directed temporal networks using PageRank centrality. *Chaos Solitons Fractals* 159(4):112107. <https://doi.org/10.1016/j.chaos.2022.112107>
29. Malviya V, Gupta GP (2016) Performance evaluation of similarity-based link prediction schemes for social network. In: *1st International Conference on Next Generation Computing Technologies*, NGCT, p 654–659. <https://doi.org/10.1109/NGCT.2015.7375202>
30. Manzali Y, Akhiat Y, Barry KA, Akachar E, Far ME (2024) Prediction of student performance using Random Forest combined with Naïve Bayes. *Comput J* 67(8):2677–2689. <https://doi.org/10.1093/COMJNL/BXAE036>
31. Mao Y, Hao Y, Cao X, Gao Y, Yao C, Lin X (2025) Boosting GNN-based link prediction via PU-AUC optimization. *IEEE Trans Knowl Data Eng*. <https://doi.org/10.1109/TKDE.2025.3525490>
32. Mohamed B, Fatima A, Soufian EA (2024) Advancing link prediction in directed social networks: a machine learning approach. In: *2024 International Conference on Circuit, Systems and Communication*, ICCSC, p 1–5. <https://doi.org/10.1109/ICCSC62074.2024.10617131>
33. Nie M, Chen D, Wang D, Chen H (2025) Local optimization policy for link prediction via reinforcement learning. *IEEE Trans Netw Sci Eng*. <https://doi.org/10.1109/TNSE.2025.3526340>
34. Oliveira GP, Moura AFC, Batista NA, Brandão MA, Hora A, Moro MM (2023) How do developers collaborate? Investigating GitHub heterogeneous networks. *Softw Qual J* 31(1):211–241. <https://doi.org/10.1007/S11219-022-09598-X>. (/TABLES/5)
35. Özkurt C (2025) Transparency in decision-making: the role of explainable AI (XAI) in customer churn analysis. *Inf Technol Econ Bus* 2(1):1–11. <https://doi.org/10.69882/ADBA.ITEB.2025011>
36. Rawashdeh A (2020) Performance based comparison between several link prediction methods on various social networking datasets (including two new methods). *Int J Adv Comput Sci Appl* 11(12):1–8. <https://doi.org/10.14569/IJACSA.2020.0111201>
37. Resce G, Zinilli A, Cerulli G (2022) Machine learning prediction of academic collaboration networks. *Sci Rep* 12(1):1–16. <https://doi.org/10.1038/S41598-022-26531-1>. (SUBJMETA=2801,530,531,639,705,766;KWRD=COMPLEX+NETWORKS,STATISTICS)
38. Rozemberczki B, Allen C, Sarkar R (2021) Multi-scale attributed node embedding. *J Complex Netw* 9(2):1–22. <https://doi.org/10.1093/COMNET/CNAB014>
39. Rui X, Zhuang J, Sun C, Wang Z (2025) Higher-order link prediction via light hypergraph neural network and hybrid aggregator. *Int J Mach Learn Cybern* 16(4):2671–2685. <https://doi.org/10.1007/S13042-024-02414-X>/FIGURES/7
40. Sellami B, Ounoughi C, Kalvet T, Tiits M, Rincon-Yanez D (2024) Harnessing graph neural networks to predict international trade flows. *Big Data Cognitive Comput* 2024 8(6):65–65. <https://doi.org/10.3390/BDCC8060065>
41. Shao H, Sun D, Wu J, Zhang Z, Zhang A, Yao S, Liu S, Wang T, Zhang C, Abdelzaher T (2020) Paper2repo: GitHub repository recommendation for academic papers. In: *The Web Conference 2020 - Proceedings*

- of the World Wide Web Conference, WWW ,p 629–639. <https://doi.org/10.1145/3366423.3380145:CTYPE.STRING:BOOK>
42. Song X, Zhang Y, Pan R, Wang H (2022) Link prediction for statistical collaboration networks incorporating institutes and research interests. *IEEE Access* 10:104954–104965. <https://doi.org/10.1109/ACCESS.2022.3210129>
 43. Thakrar K, & Chauhan A (2025) GitHub Stargazers | Building Graph- and Edge-level Prediction Algorithms for Developer Social Networks. Preprint at <https://arxiv.org/pdf/2502.00058>
 44. Ting TJ, Li X, Sanner S, Abdulhai B (2021) Revisiting random forests in a comparative evaluation of graph convolutional neural network variants for traffic prediction. In: *IEEE International Intelligent Transportation Systems Conference (ITSC)*, p 1259–1265. <https://doi.org/10.1109/ITSC48978.2021.9564595>
 45. Verma AK, Saxena R, Jadeja M, Bhateja V, Lin JCW (2023) Bet-GAT: an efficient centrality-based graph attention model for semi-supervised node classification. *Appl Sci* 13(2):847–847. <https://doi.org/10.3390/APP13020847>
 46. Xiang Y, Fujimoto K, Schneider J, Jia Y, Zhi D, Tao C (2019) Network context matters: graph convolutional network model over social networks improves the detection of unknown HIV infections among young men who have sex with men. *J Am Med Inform Assoc* 26(11):1263–1271. <https://doi.org/10.1093/JAMIA/OCZ070>
 47. Xu Z, Chen W, Zou Y, Fang Z, Wang S (2024) Attention-based stackable graph convolutional network for multi-view learning. *Neural Netw* 180:106648. <https://doi.org/10.1016/J.NEUNET.2024.106648>
 48. Yao L, Wang L, Pan L, Yao K (2016) Link prediction based on common-neighbors for dynamic social network. *Procedia Comput Sci* 83:82–89. <https://doi.org/10.1016/J.PROCS.2016.04.102>
 49. Zhang J, Li M, Gao K, Meng S, Zhou C (2021) Word and graph attention networks for semi-supervised classification. *Knowl Inf Syst* 63(11):2841–2859. <https://doi.org/10.1007/S10115-021-01610-3>. (TABLES/5)
 50. Zhou H, Ravi H, Muniz CM, Azizi V, Ness L, de Melo G, Kapadia M (2020) GitEvoIve: predicting the evolution of github repositories. Preprint at <https://arxiv.org/pdf/2010.04366>
 51. Zhou Z, Wan G, Du B (2025) Common neighbor completion with information entropy for link prediction in social networks. *Data Sci Eng* 10(1):40–53. <https://doi.org/10.1007/S41019-024-00267-6/FIGURES/6>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Cihan Bayraktar obtained his Bachelor's degree in 2006 from the Department of Computer and Instructional Technologies Education at Marmara University, Türkiye, his Master's degree from the Department of Business Administration at the Institute of Social Sciences at Karabük University, Türkiye, and his Ph.D. from the Department of Management Information Systems at the Institute of Information Technology at Gazi University, Türkiye. After completing his bachelor's degree, he worked as an Information Technology Teacher. In February 2011, he began working as a Lecturer in the Information Security Technology Programme at Karabük University Eskipazar Vocational School, continuing until January 2023. He then began working as an Assistant Professor in the same unit and programme, where he continues to work. His research interests include Windows Python applications, Databases, Information Security, Data Mining, Cyber Security, Artificial Intelligence, Data Management, Anomaly Detection, Social Network Analyses and Industry 4.0 Applications.